Software Architecture and Design Specification

Project: Real-Time Collaborative Virtual Whiteboard

Team: Team 11 — Rachit S Mane, P Chinni Krishna Reddy, Pranav V Menon, Prithvi W

Version: 1.0

Date: 11-10-2026

Status: Final for submission — written against SRS v1.0; team sign-off pending (see Approvals); written as a design before implementation (historical); implemented and tested in the execution of 12-10-2026 (uncommitted working tree on base commit 82af885; see `docs/IMPLEMENTATION_STATUS.md`); updated 02-11-2026 to the implementation on `main` after PR #33 (API-02 board rename, API-01 `board:renamed` and `notice`, §4.5–§4.6)

Revision History

Version	Date	Author	Change Summary
0.1	10-10-2026	Krishna, Rachit, Prithvi (draft)	First complete draft for team review
0.2	10-10-2026	Krishna, Rachit, Prithvi (draft)	Acknowledgement/reconciliation protocol and reconnection settings (§4.1); ownership-transfer endpoint and SD-04; snapshot size policy and FR-008; security control mapping; test hooks
0.3	10-10-2026	Krishna, Rachit, Prithvi (draft)	Board limit defined on BSON size with conservative accounting; clear/restore never completed without their version (restore writes the version first, roll-forward on start-up); sequence diagrams split into legible parts, SD-01 adds reconnection, SD-04 adds all rejection cases; component diagram shows Version History → Board & Access and Board & Access → Real-time dependencies; numbered figures
0.4	10-10-2026	Krishna, Rachit, Prithvi (draft)	Size measures M1–M4 with version-document margin derivation; restore consistency and recovery design with mandatory start-up order (roll-forward before sequence recovery); component diagram split into overview and server-dependency figures; `FAULT_INJECT` test hook; message limit stated in bytes
1.0	11-10-2026	Krishna, Rachit, Prithvi	Final for submission: durable per-board journal (`pendingOp`) for clear and restore with start-up recovery; idempotent restore (`Idempotency-Key`); new SD-05 (clear); SD-02 redrawn; size bound checked by computing the encoded worst case (`tools/size_model.py`; a calculation, not a test of the system); selected technology stack; security event record (§4.4, D13b); restore pre-commit checks, failed start-up recovery and retry/concurrency rules made explicit (§3.6, API-03, SD-02, SD-05); failed logins without account enumeration and `subjectId` defined (§3.9.6, §4.4)

Approvals

Role	Name	Signature/Date
SAD Lead	P Chinni Krishna Reddy	
SRS Lead	Rachit S Mane	
STP Lead	Pranav V Menon	
Security & Traceability Lead	Prithvi W	

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Real-Time Collaborative Virtual Whiteboard. It explains how the requirements of the SRS are realised by components,

interfaces, data stores and protocols, and records the main design decisions and their rationale.

1.2 Scope

The document covers the browser client, the application server (request interface, real-time collaboration, access control and version history), the database, and the cross-cutting concerns of security, error handling, logging and monitoring for version 1.0 as scoped in SRS §1.2. Technologies named here are the selected stack for version 1.0 (DECISIONS.md D10).

1.3 Audience

Team 11 developers and testers, the course instructor and teaching assistants, and reviewers assessing security and maintainability.

1.4 Definitions

Terms defined in SRS §1.4 apply. Additional terms: **ADR** — architecture decision record; **JWT** — JSON Web Token; **Room** — the Socket.IO group of connections attached to one board; **STRIDE** — Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege; **SPA** — single-page application; **ODM** — object-document mapper.

2. Document Overview

2.1 How to use this document

Section 3 describes the architecture: goals, stakeholders, components (COMP-xx), the chosen pattern, technology, risks, requirement traceability and security architecture. Section 4 describes the design: sequence diagrams (SD-xx), API contracts (API-xx), error handling, logging, monitoring and UX. Every component and API lists the SRS requirement IDs it realises.

2.2 Related Documents

- Team 11 SRS v1.0 (requirement IDs FR-, NFR-, SEC-OBJ-, SEC-REQ-).
- Team 11 STP v1.0 (test case IDs TC-).
- Requirements Traceability Matrix (`docs/traceability/rtm.md`) and ID registry.
- Decision Log (`docs/DECISIONS.md`).

3. Architecture

3.1 Goals & Constraints

Goals:

- **Consistency:** every participant converges on the same board state, defined by server order (FR-021 to FR-023).
- **Responsiveness:** operations visible to others within the target of NFR-001; local edits shown immediately.
- **Security by default:** every request and real-time message authenticated and authorised on the server (SEC-REQ-002, SEC-REQ-003).
- **Recoverability:** no acknowledged operation is lost (FR-027); clients resynchronise after disconnection (FR-025, NFR-004).
- **Simplicity:** an architecture that a four-person student team can build, test and explain within the course.

Constraints:

- Single server instance for version 1.0 (SRS §2.5); no horizontal scaling.
- Selected stack: MongoDB, Express.js, React, Node.js, Socket.IO (D10).
- Desktop browsers only (NFR-006).
- MongoDB's 16 MiB maximum document size limits one stored snapshot; v1.0 therefore limits board content to 15 MiB (FR-008, §3.6).

3.2 Stakeholders & Concerns

Stakeholder	Main concerns
Board owners	Control over who can see and change a board; ability to recover earlier content.
Editors and viewers	Low latency, no lost work, clear feedback when something is rejected or the connection drops.
Guests	Joining quickly without an account; clear notice when access ends.
Developers (Team 11)	Modularity, clear contracts between client and server, testability.
Testers (Team 11)	Observable behaviour, measurable targets, test hooks (logs, metrics, health check).
Instructor / assessors	Traceability to requirements, documented rationale, security analysis.

3.3 Component (UML) Diagram

The template placeholder mentions a class diagram; the assignment brief requires a **UML component diagram**, which is provided here in two parts so that its labels stay legible:

- **Figure 1 (overview)** shows the client and server packages, the two external interfaces — API-01 (Socket.IO events) and API-02/API-03 (REST over HTTPS) — as provided (lollipop) and required (socket) interfaces, and the server layers. The four service components are grouped in one box; their individual dependencies are shown in Figure 2.

- **Figure 2 (server dependencies)** shows the labelled dependencies between the server components, including Version History → Board & Access (`authorize`, SD-02) and Board & Access → Real-time Collaboration Server (role changes and revocations, SD-03, SD-04). Every server component also writes logs and metrics to COMP-09 (§4.4); these arrows are omitted for readability.

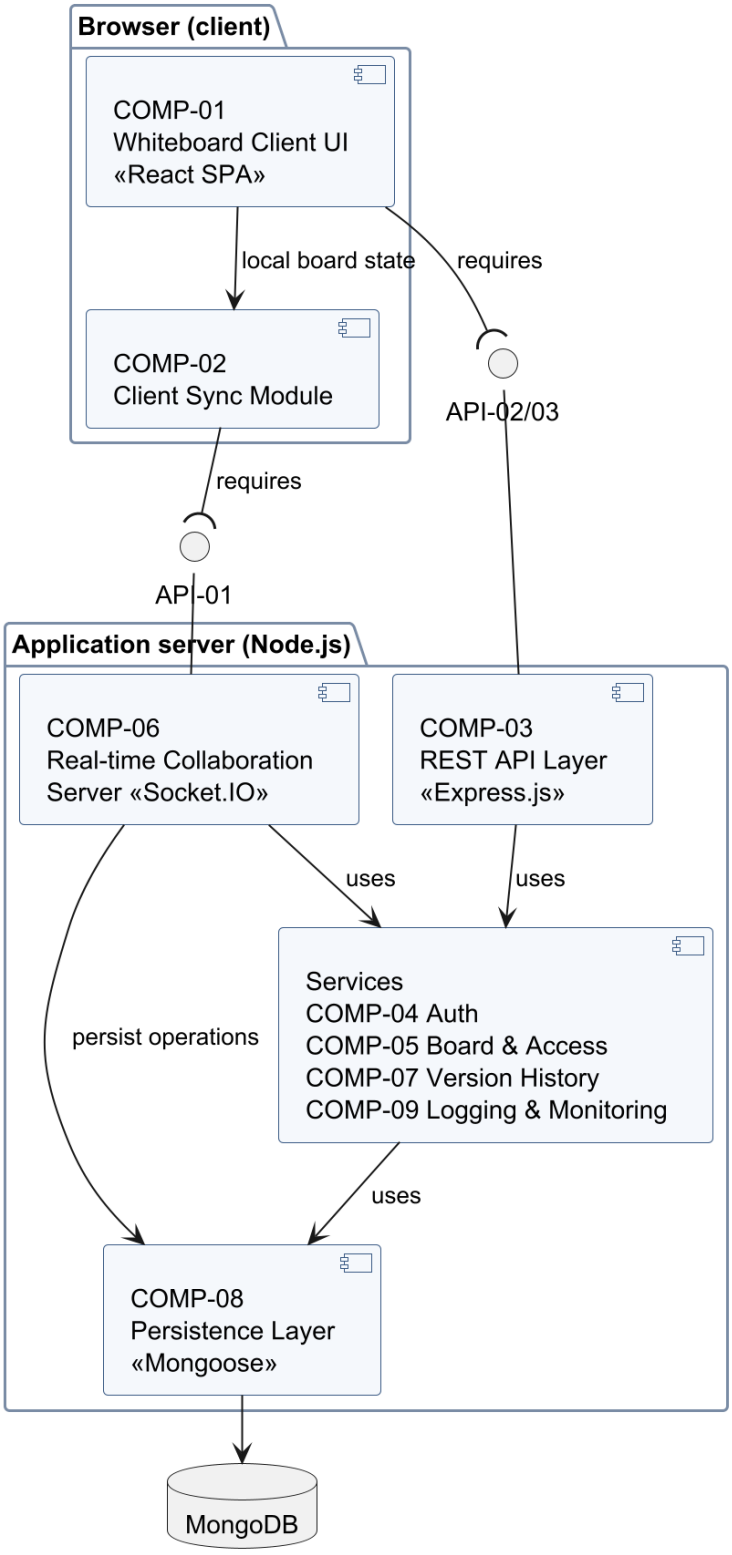


Figure 1 — UML component diagram, part 1: overview

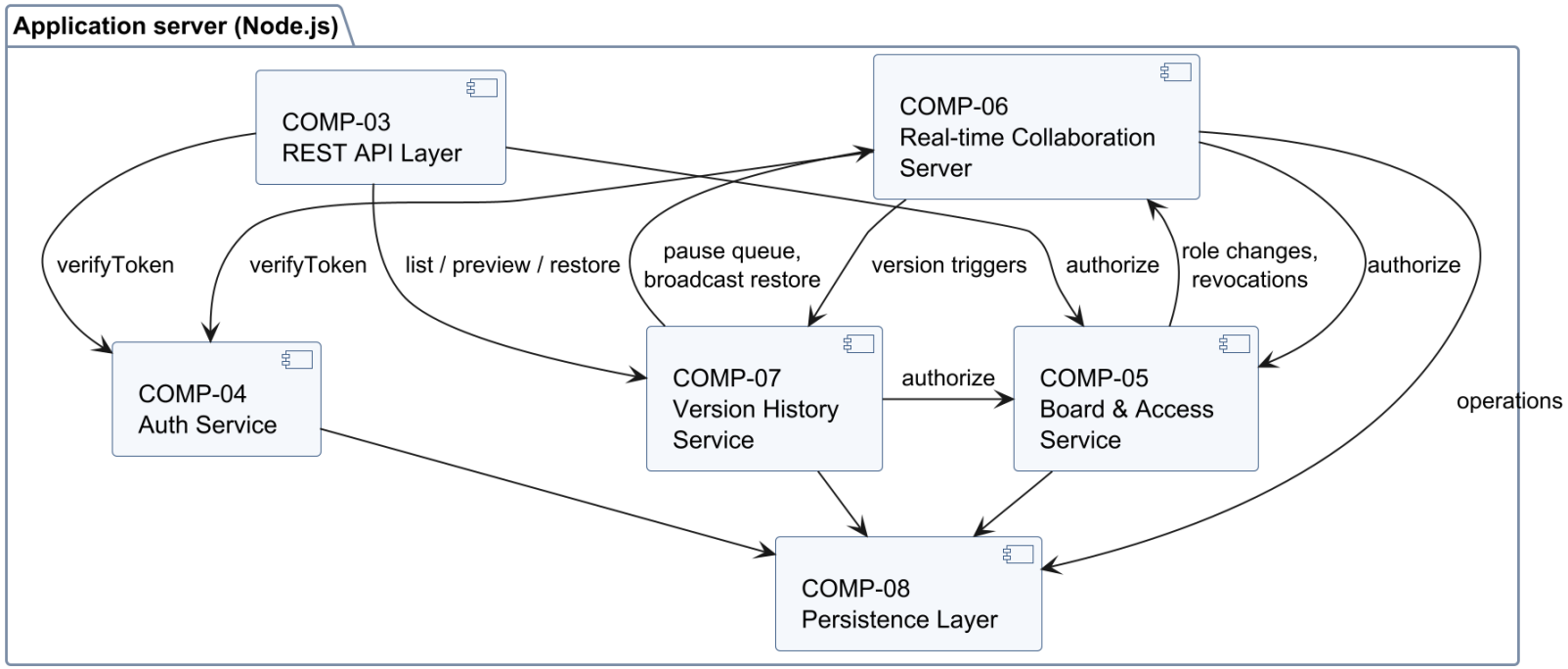


Figure 2 — UML component diagram, part 2: dependencies between server components

3.4 Component Descriptions

ID	Component	Responsibilities	Provided interfaces	Realises
COMP-01	Whiteboard Client UI	React single-page application: dashboard, board canvas rendering, toolbar, participant panel, version panel, sharing and members dialogs (including ownership transfer with confirmation); role-aware enabling of controls; error, board-full and connection-status messages; `board-ready` performance mark.	Browser UI	FR-001–FR-007, FR-031, FR-032, FR-043, FR-052, NFR-003, NFR-006, NFR-007
COMP-02	Client Sync Module	Keeps `confirmed` state, `pending` own operations and an out-of-order `buffer` (§4.1.3); renders optimistically; applies acknowledgements and `op:applied` strictly by sequence number; repairs gaps with `board:join`; rolls back rejected operations; reconnects with bounded backoff and resynchronises (§4.1.4).	Board state store used by COMP-01	FR-020–FR-026, NFR-001, NFR-004
COMP-03	REST API Layer	Express.js routes for API-02 and API-03; middleware chain for token verification, schema validation, rate limiting, error formatting and request IDs.	API-02, API-03 (HTTPS, JSON)	FR-040–FR-050, SEC-REQ-002, SEC-REQ-005, SEC-REQ-006
COMP-04	Auth Service	Registration, password hashing and verification, issuing and verifying session tokens, logout revocation list, guest tokens derived from share links.	`verifyToken`, `issueSession`, `issueGuestToken` (internal)	FR-040, FR-041, FR-046, SEC-REQ-001, SEC-REQ-002
COMP-05	Board & Access Service	Boards, members and share links; the permission policy (single implementation of the FR-051 matrix) used by every other server component; link creation, expiry and revocation; ownership transfer (API-02, SD-04); board deletion.	`authorize(actor, boardId, action)`, board/link/member operations (internal)	FR-042–FR-051, SEC-REQ-003, SEC-REQ-004

ID	Component	Responsibilities	Provided interfaces	Realises
COMP-06	Real-time Collaboration Server	Socket.IO server: connection authentication, one room per board, per-board task queue that serialises operations, clear, restore and version tasks, validation and authorisation of every event, board size limit, sequence-number assignment, persistence before acknowledgement and broadcast (§4.1.2), presence, role-change notifications and forced disconnects on revocation, removal, deletion and ownership transfer.	API-01 (Socket.IO events)	FR-001–FR-008, FR-020–FR-027, FR-047–FR-050, FR-052, NFR-001, NFR-002, SEC-REQ-003, SEC-REQ-005, SEC-REQ-006
COMP-07	Version History Service	Automatic version triggers (interval *T*, before clear, last participant leaves) with the no-change rule; version list and preview; owner-only, journalled and idempotent restore that always creates a new version; retention of the 20 newest; defensive snapshot size check; hand-off of restored state to COMP-06 for broadcast.	API-03 operations, `snapshot(boardId, trigger)` (internal)	FR-007, FR-030–FR-035, NFR-005
COMP-08	Persistence Layer	MongoDB database and Mongoose models/repositories for users, boards, board objects, memberships, share links, versions and revoked tokens; indexes and atomic updates.	Repository functions (internal)	FR-007, FR-027, FR-033 (journal and start-up recovery), FR-035, FR-044, SEC-REQ-001, SEC-REQ-004
COMP-09	Logging & Monitoring	Structured JSON logging with request and correlation IDs, security event log, metrics (operation latency, connections, rejections, restores), health-check endpoint.	`log`, `metric` (internal); `GET /api/v1/health`	SEC-REQ-008, NFR-001, NFR-004 (measurement support)

3.5 Chosen Architecture Pattern and Rationale

Pattern: client–server with a **layered server** (interface layer → service layer → persistence layer) combined with **event-driven publish/subscribe** for real-time collaboration (one Socket.IO room per board) and a **server-authoritative state** model.

- The interface layer (COMP-03 for REST, COMP-06 for Socket.IO) authenticates, validates and authorises every input before calling services.
- The service layer (COMP-04, COMP-05, COMP-07, and the sequencing logic of COMP-06) contains the business rules; the permission policy exists in exactly one place (COMP-05).
- The persistence layer (COMP-08) hides MongoDB behind repository functions.

- The server orders every operation (sequence numbers) and is the only source of truth; clients render optimistically but always converge to server order.

Why this pattern:

Concern	How the pattern addresses it
Consistency (FR-021–FR-023)	One authority orders all operations; per-property last-write-wins is deterministic because order is global per board.
Security (SEC-REQ-003)	All changes pass through the server, where role checks use server-side data only.
Latency (NFR-001)	Persistent Socket.IO connections avoid per-operation HTTP overhead; rooms limit broadcast to one board.
Team size and testability	Layers can be developed and unit-tested separately; REST and Socket.IO contracts are explicit (§4.3).

Alternatives considered and rejected:

Alternative	Reason for rejection
Peer-to-peer (WebRTC mesh)	No central authority for ordering and permission checks; harder to secure and persist.
CRDT or operational transformation	Merges concurrent edits more precisely but adds significant complexity; per-property LWW is sufficient for strokes, shapes and short text (limitation recorded in SRS §4.2).
Microservices	Operational overhead disproportionate to a single-instance student project.
REST polling for updates	Cannot meet NFR-001 efficiently; higher server load.

3.6 Technology Stack & Data Stores

Selected technology stack (DECISIONS.md D10):

Layer	Technology	Purpose
Client	React (SPA) with the HTML Canvas 2D API	UI, rendering of board objects
Real-time transport	Socket.IO (server and client libraries)	Board events, presence, notifications (API-01)
Server runtime	Node.js (current LTS release)	Application server
HTTP framework	Express.js	REST API (API-02, API-03), middleware
Database	MongoDB (Community edition, local) with Mongoose ODM	Persistent data
Authentication	JWT session tokens; bcrypt password hashing (D13)	SEC-REQ-001, SEC-REQ-002
Validation	JSON-schema validation of REST bodies and Socket.IO payloads	SEC-REQ-005

Data stores (MongoDB collections):

Collection	Key fields	Notes
`users`	`_id`, `email` (unique index), `passwordHash`, `displayName`, `createdAt`	No plain-text password (SEC-REQ-001)
`boards`	`_id`, `title`, `ownerId`, `seq` (last committed sequence number), `contentBytes`, `pendingOp`, `updatedAt`	`seq` advanced once per accepted operation (§4.1.2); `contentBytes` = M3 (FR-008); `pendingOp` = durable journal of an unfinished clear or restore (see below)
`boardObjects`	`boardId`, `objectId`, `kind`, geometry, `color`, `strokeWidth`, `text`, `updatedSeq`	Unique index (`boardId`, `objectId`); current state only. Additions and updates are written through the MongoDB driver directly, not through Mongoose document casting, after the payload has passed the operation schemas (SEC-REQ-005: no additional properties, whitelisted update fields); the stored fields and BSON types are the same as the Mongoose model's (`server/tests/integration/persistence.test.js`). Deletions, `boards` and `versions` writes use Mongoose.
`memberships`	`boardId`, `userId`, `role` (`editor`/`viewer`)	The Owner is stored on the board, not here
`shareLinks`	`_id`, `boardId`, `tokenHash`, `role`, `expiresAt`, `revokedAt`, `createdBy`	Token stored only as SHA-256 hash (SEC-REQ-004); TTL index for cleanup
`versions`	`boardId`, `number`, `boardSeq`, `trigger`, `restoredFrom`, `requestId`, `createdAt`, `createdBy`, `objects[]`	Full snapshot; unique index (`boardId`, `number`); sparse unique index (`boardId`, `requestId`) for idempotent restores; `boardSeq` implements the no-change rule
`revokedTokens`	`jti`, `expiresAt`	Logout revocation list; TTL index removes entries after expiry

Database prerequisites. Version 1.0 runs one MongoDB server without a replica set, so multi-document transactions are **not** used. The design relies only on guarantees a standalone server provides: every single-document write is atomic, and writes issued with write concern `{w: 1, j: true}` are durable in the journal before they are acknowledged. Journaling must be enabled (the WiredTiger default). Running a single-node replica set would allow transactions, but the recovery design below does not depend on them.

Version storage, triggers and retention (behaviour per SRS §4.3):

- *Storage:* each version is one document containing a full copy of the board's objects (`objects[]`) and the board `seq` it corresponds to (`boardSeq`). Each element of `objects[]` is the object in **snapshot form** — `id`, `kind`, geometry, `color`, `strokeWidth`, `text` — without the storage-only fields `_id`, `boardId` and `updatedSeq` of `boardObjects`.
- *Triggers:* (a) every T = 5 minutes (configuration `VERSION_INTERVAL_MS`, D11a), (b) before a clear, (c) when the last participant leaves. Each automatic trigger creates a version only if `boards.seq` differs from the latest version's `boardSeq` (no-change rule). A restore always creates a version with `trigger = "restore"` and `restoredFrom = n`.

- *Retention:* after each completed version write, versions older than the 20 most recent are deleted (FR-035), except a version referenced by `pendingOp`. No other operation modifies or deletes a version.

Serialisation. Every change to a board — real-time operations, clear, restore, automatic version creation and ownership transfer — runs as a task in that board's single FIFO queue in COMP-06 (one task at a time per board). A REST restore request is enqueued as such a task and its HTTP response is sent when the task finishes. Start-up recovery (below) completes before the server accepts any connection or request. The design assumes exactly one server instance; several instances would need a distributed lock and are out of scope (§4.6).

Durable operation journal (`boards.pendingOp`). Clear and restore are the two operations that change many documents. Before their first destructive write, the task records the operation in `boards.pendingOp` with one atomic, journaled single-document update; the same update that sets the new `seq` at the end also removes `pendingOp`. The **commit point** is the journaled write of `pendingOp`: from then on the operation is always completed (never rolled back), either by the running task or by start-up recovery. No client receives an acknowledgement, `op:applied`, `board:state` or HTTP success before `pendingOp` has been removed.

Step	Clear (task for `board:clear`, FR-007)	Restore of version *n* (API-03, FR-033)
1	If the board is empty: acknowledge `{ok: true, seq: null, noop: true}`; stop.	If a version with the request's `requestId` exists for this board: if its `restoredFrom` is *n*, return the recorded result `{newVersion: number, restoredFrom, seq: boardSeq}` without applying anything again (idempotent retry); otherwise reject with `409 IDEMPOTENCY_KEY_REUSED`; stop. Load snapshot *n* (`VERSION_NOT_FOUND` if not retained).
2	Apply the "before clear" trigger: write version *v* if the board has changed content (no-change rule). On failure: reject with `INTERNAL`; nothing else changed.	Compute M4 of version *r* (snapshot *n* plus metadata); above the defensive threshold, reject with `INTERNAL` and log `version.snapshot_too_large` (should not occur, FR-008).
3	**Commit:** set `pendingOp = {type: "clear", seq: s, preClearVersion: v}` where `s = seq + 1` (journaled).	**Commit:** set `pendingOp = {type: "restore", seq: s, version: r, restoredFrom: n, requestId}` where `s = seq + 1` and `r` is the next version number (journaled).
4	Delete all `boardObjects` of the board.	Insert version *r* (`trigger: "restore"`, `restoredFrom: n`, `boardSeq: s`, `requestId`) unless it already exists.
5	—	Delete all `boardObjects` of the board; insert the snapshot objects with `updatedSeq = s`.
6	**Complete:** set `seq = s`, `contentBytes = 0`, unset `pendingOp` (one atomic, journaled update).	**Complete:** set `seq = s`, `contentBytes` = M3 of the snapshot, unset `pendingOp` (one atomic, journaled update); apply retention.
7	Acknowledge the submitter `{ok: true, seq: s}`; broadcast `op:applied` (type `board:clear`).	Broadcast `board:state` and `version:restored`; respond `201 {newVersion: r, restoredFrom: n, seq: s}`.

Failure handling and recovery (D5c):

Failure point	State left behind	What happens
Before step 3, or the commit write itself fails (including a failed before-clear version, a missing snapshot or a refused size check)	Board unchanged; at most an extra "before clear" version identical to the current content	Request rejected (`INTERNAL` or `VERSION_NOT_FOUND`); the extra version is harmless (its `boardSeq` equals `seq`, so the no-change rule is unaffected).
After step 3, before step 6 (server crash)	`pendingOp` present; objects partly or fully deleted (clear) or partly replaced (restore); version *r* may or may not exist	Start-up recovery completes the operation from `pendingOp`, repeating steps 4–6. Every step is idempotent: deleting already-deleted objects changes nothing; version *r* is inserted only if missing; the object set is replaced as a whole.
After step 6, before or during step 7 (crash, or the acknowledgement or HTTP response is lost)	Operation complete; `pendingOp` absent	Nothing is repeated. Clients resynchronise from the full state (FR-025) and see the result; the Owner's restore retry with the same key returns the recorded result.
During start-up recovery (crash or database error)	`pendingOp` still present	A crash: recovery runs again on the next start-up with the same result, because steps 4–6 are idempotent. A database error: retried up to 3 times; then `recovery.required` is logged with the board ID and the server exits with a non-zero status without accepting any connection, so no client ever sees a half-completed board; recovery is repeated at the next start.
A database error (not a crash) during steps 4–6	`pendingOp` present	The task retries the remaining steps up to 3 times; if they still fail, the server logs `recovery.required`, stops accepting new tasks for that board (operations are rejected with `INTERNAL`) and completes the operation at the next start-up. It never reports success.

Start-up recovery order (for every board, before the server accepts any connection or request):

1. *Complete journalled operations:* if `pendingOp` is set, repeat steps 4–6 of its type (for a restore, re-create version *r* from snapshot `restoredFrom` if missing). The snapshot is still available because retention never deletes a version referenced by `pendingOp`. Log `board.clear_rolled_forward` or `version.restore_rolled_forward`.
2. *Sequence recovery:* set `boards.seq` to the maximum of `boards.seq` and the largest `updatedSeq` in `boardObjects` (covers a crash between the two writes of an ordinary operation, §4.1.2). Step 1 runs first so that objects written by an unfinished restore cannot raise `seq` past an incomplete operation.
3. *Content size:* recompute `contentBytes` (M3) from `boardObjects`.

Duplicate requests and client view.

- *Distinct restores:* every restore request carries an `Idempotency-Key` (a new UUID per user action, API-03). A request with a new key is a distinct restore and always creates a new version, even if the same version was restored before.
- *Retries:* a retry with the same key and the same version *n* returns the recorded result (same `newVersion`, `restoredFrom` and `seq`) and applies nothing again. Because tasks of a board run in FIFO order, a retry that arrives while the original is still queued or running is processed after it and therefore sees its result. A failure before the commit point writes no `requestId`, so a later retry with that key performs the restore normally.
- *Key reuse:* the same key with a different version number is rejected with `409 IDEMPOTENCY_KEY_REUSED`; nothing changes.
- *Concurrency:* concurrent restores (and clears) of one board are serialised by the board queue; each distinct request gets its own version number and sequence number in queue order.
- *Limit:* the key is remembered through the `requestId` of version *r*; once retention (FR-035) has removed version *r*, the key is no longer recognised. Clients retry only immediately after a failed or interrupted request, so this does not arise in normal use.
- *Clear:* a clear has no idempotency key. A clear repeated after it was applied finds an empty board and is a no-op; the client never resubmits a clear automatically (pending operations are discarded after resynchronisation, §4.1.4), so a second clear always needs a new Owner confirmation.
- *Client view:* a client whose request was interrupted receives no success; after reconnection it shows the server state, and the Owner's client retries an interrupted restore once with the same key to learn the outcome.

Logged events (§4.4): `op.clear_failed`, `version.restore_failed`, `board.clear_rolled_forward`, `version.restore_rolled_forward`, `recovery.required`.

These mechanisms are a design specification; they are verified by TC-024 (restore) and TC-025 (clear). Historical v1.0 note: when this was written the system did not exist and neither test had been executed; both were executed with status Pass on 12-10-2026 (STP Appendix A, `docs/IMPLEMENTATION_STATUS.md`).

Size measures (used consistently in SRS FR-008, SEC-REQ-005 and STP TC-022, TC-107):

Measure	Definition	Limit	Enforced by
M1 — message size	Bytes of one Socket.IO packet as received by the server (event name, acknowledgement id, framing and JSON payload)	64 KiB = 65,536 bytes (`maxHttpBufferSize`)	Socket.IO engine before any handler runs (SEC-REQ-005)
M2 — object size	`BSON.calculateObjectSize(object)` of one board object in snapshot form (`id`, `kind`, geometry, `color`, `strokeWidth`, `text`)	none on its own; bounded in practice by M1	Computed by COMP-06 for every add/update
M3 — content size	Sum over all objects of (M2 + 9 bytes); the 9 bytes cover the array element header (type byte, index key of up to 7 digits, terminator) and over-estimate it for indices below 10^6	15 MiB = 15,728,640 bytes (FR-008)	COMP-06, before any write (`BOARD_FULL`)
M4 — version document size	`BSON.calculateObjectSize` of the complete `versions` document (metadata + `objects[]`)	MongoDB maximum 16 MiB = 16,777,216 bytes; defensive refusal above 16 MiB − 64 KiB = 16,711,680 bytes	COMP-07, before inserting a version

M1 and M3 are independent: a single message can never exceed 65,536 bytes, while M3 accumulates over many operations. JSON (M1) and BSON (M2) sizes of the same object differ — BSON stores each coordinate as an 8-byte double with a per-element header — therefore neither size is inferred from a point count; TC-022 measures both.

Bound of M4 (derivation): a version document consists of the document header and terminator (5 bytes); the metadata fields `_id` (ObjectId, 17 bytes), `boardId` (ObjectId, 21), `number` (int32, 12), `boardSeq` (int64, 18), `trigger` (string of at most 16 characters, ≤ 30), `restoredFrom` (int32 or null, ≤ 18), `createdAt` (date, 19), `createdBy` (ObjectId or guest ID of at most 64 characters, ≤ 80) and `requestId` (UUID string or null, ≤ 52) — at most 267 bytes; the `objects` field header (9 bytes) and the array's own header and terminator (5 bytes); and the elements. The elements occupy at most M3 bytes because M3 counts every element header with 9 bytes, the maximum for indices below 10^7 (a 15 MiB board cannot hold that many objects: the smallest object is larger than 30 bytes). Hence **M4 ≤ M3 + 286 bytes ≤ 15,728,926 bytes**, which is 982,754 bytes below the defensive threshold and 1,048,290 bytes below MongoDB's limit. COMP-06 therefore rejects every operation that would break the bound before anything is stored, acknowledged or broadcast, and COMP-07 additionally measures M4 before every insert. The derivation assumes that versions store objects in snapshot form only and that the schema enforces the metadata bounds above; `tools/size_model.py` reproduces this calculation with a BSON encoder (a design calculation, not a measurement of the application), and TC-022 measures the real system.

Snapshot size (version 1.0 vs. future):

Aspect	Version 1.0 (specified)	Future option (not in v1.0)
Maximum board content	M3 ≤ 15 MiB (15,728,640 bytes), enforced by COMP-06 on every add/update before anything is written (`BOARD_FULL`, FR-008, D11c). `contentBytes` increases on add, changes by the size difference on update, decreases on delete and clear, and is recomputed from `boardObjects` at start-up.	Higher limit with chunked storage
Snapshot storage	One MongoDB document per version; M4 ≤ M3 + 286 bytes (derivation above)	Chunked snapshots (e.g. GridFS)
Snapshot larger than allowed	Prevented by the M3 limit. Defensive check: above 16,711,680 bytes the version is not written and `version.snapshot_too_large` is logged; for a periodic or last-participant trigger the Owner gets an error notice and the live board continues unchanged; for a before-clear trigger the clear is rejected; for a restore the restore is rejected	—
Reference size	TB-STD is estimated at about 2 MiB BSON (300 strokes × 200 points; see `tools/size_model.py` for the calculated value)	—

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Single server instance is a single point of failure	Board unavailable during outages	Persistence before broadcast (FR-027); automatic client reconnection (FR-025); scaling with a Socket.IO adapter recorded as future work (§4.6).
Snapshot exceeds MongoDB 16 MiB document limit	Version creation would fail	Prevented in v1.0 by the 15 MiB board content limit (FR-008) and a defensive size check before writing; `snapshot_bytes` metric; chunked storage is future work (§4.6).
LWW overwrites a concurrent change to the same object	User's edit silently replaced	Documented limitation (SRS §4.2); updates carry complete property values so every client converges; candidate future CRDT.
Performance targets prove infeasible when measured	NFRs fail	Latency harness (TC-019) built in the first implementation sprint; if a target cannot be met, a change request updates NFR-001 to NFR-005.
Leaked share link	Unauthorised access	Expiry, revocation with forced disconnect, hashed storage, Owner-visible link list (§3.9).
Team unfamiliar with canvas rendering or Socket.IO	Schedule slip	Spike prototypes before implementation; pair work on COMP-02/COMP-06.
Clock differences distort latency measurement	Misleading NFR-001 results	Measure with all clients in one process or one machine (SRS §5 test conditions).

3.8 Traceability to Requirements

Every SRS requirement is realised by at least one component. The component column must match the "Module" column, and the design elements must match the "Design Spec" part, of the master RTM; `tools/check_trace.py` verifies this.

Req ID	Requirement short	Component(s)	Design elements
FR-001	Freehand stroke	COMP-01, COMP-02, COMP-06	SD-01, API-01
FR-002	Colour and stroke width	COMP-01	API-01
FR-003	Rectangle, ellipse, line	COMP-01, COMP-02, COMP-06	API-01
FR-004	Text objects	COMP-01, COMP-02, COMP-06	API-01
FR-005	Select, move, restyle, delete	COMP-01, COMP-02, COMP-06	API-01
FR-006	Object eraser	COMP-01, COMP-02, COMP-06	API-01
FR-007	Owner-only clear; pre-clear content kept; recovery	COMP-05, COMP-06, COMP-07, COMP-08	SD-05, API-01
FR-008	Board content size limit 15 MiB (M3)	COMP-01, COMP-06	SD-01, API-01
FR-020	Broadcast accepted operations	COMP-02, COMP-06	SD-01, API-01
FR-021	Server sequence numbers and ordering	COMP-02, COMP-06	SD-01, API-01
FR-022	Independent edits preserved	COMP-06	SD-01
FR-023	Same-object last write wins	COMP-02, COMP-06	SD-01
FR-024	Full state on join	COMP-06, COMP-08	SD-03, API-01
FR-025	Disconnect indicator and resync	COMP-02, COMP-06	SD-01, API-01
FR-026	Rejection feedback and rollback	COMP-02, COMP-06	SD-01, API-01
FR-027	Persist before broadcast	COMP-06, COMP-08	SD-01
FR-030	Automatic versions, no-change rule	COMP-07, COMP-08	API-03
FR-031	Version list for all roles	COMP-01, COMP-07	API-03
FR-032	Read-only version preview	COMP-01, COMP-07	API-03
FR-033	Owner-only restore as new version; recovery	COMP-05, COMP-07, COMP-08	SD-02, API-03
FR-034	Restored state pushed to all	COMP-06, COMP-07	SD-02, API-01
FR-035	Retain at least 20 versions	COMP-07, COMP-08	API-03
FR-040	Register account	COMP-03, COMP-04	API-02
FR-041	Log in and log out	COMP-03, COMP-04	API-02
FR-042	Create board, creator is Owner	COMP-03, COMP-05	API-02
FR-043	Dashboard of boards	COMP-01, COMP-03, COMP-05	API-02
FR-044	Owner-only board deletion	COMP-05, COMP-06	API-02
FR-045	Create share link with role and expiry	COMP-05	API-02
FR-046	Join via link (member or guest)	COMP-04, COMP-05	SD-03, API-02
FR-047	Revoke share link	COMP-05, COMP-06	SD-03, API-02
FR-048	Expired link rejected	COMP-05, COMP-06	SD-03
FR-049	Change member role or remove	COMP-05, COMP-06	API-02
FR-050	Transfer ownership to a member	COMP-03, COMP-05, COMP-06	SD-04, API-02
FR-051	Permission matrix enforced	COMP-03, COMP-05, COMP-06	SD-01, SD-02, SD-03, SD-04, SD-05
FR-052	Participant presence list	COMP-01, COMP-06	API-01
NFR-001	95% of updates visible within 500 ms	COMP-02, COMP-06, COMP-09	SD-01
NFR-002	At least 10 concurrent participants	COMP-06	—
NFR-003	TB-STD loads within 3 s	COMP-01, COMP-03	—
NFR-004	Reconnect recovery within 5 s	COMP-02, COMP-06	SD-01
NFR-005	Restore visible to all within 3 s	COMP-06, COMP-07	SD-02
NFR-006	Chrome, Firefox, Edge support	COMP-01	—

Req ID	Requirement short	Component(s)	Design elements
NFR-007	Role/status visible, labelled toolbar	COMP-01	—
SEC-REQ-001	Password policy and salted hashing	COMP-04, COMP-08	API-02
SEC-REQ-002	Tokens required, expiry, logout	COMP-03, COMP-04, COMP-06	SD-03, API-01, API-02
SEC-REQ-003	Server-side authorisation of every request/event	COMP-03, COMP-05, COMP-06	SD-01, SD-02, SD-03, SD-04, SD-05
SEC-REQ-004	Link token entropy, hashing, expiry	COMP-05, COMP-08	SD-03, API-02
SEC-REQ-005	Schema validation, size limit, no HTML	COMP-01, COMP-03, COMP-06	API-01, API-02, API-03
SEC-REQ-006	Login and operation rate limits	COMP-03, COMP-06	API-01, API-02
SEC-REQ-007	TLS 1.2+ in deployment	COMP-03, COMP-06	—
SEC-REQ-008	Security event logging without secrets	COMP-03, COMP-04, COMP-05, COMP-06, COMP-07, COMP-09	SD-04

Reverse view: each component lists the requirements it realises in 3.4, and each API lists its requirements in 4.3.

3.9 Security Architecture

All controls below were specified as design decisions (D13) before any of them was implemented (historical v1.0 note). They are now implemented; a control counts as verified only through its security test cases: TC-101 to TC-108 Pass, TC-109 Blocked (no HTTPS deployment), so the transport controls of SEC-REQ-007 are unverified (execution of 12-10-2026, `docs/IMPLEMENTATION_STATUS.md`).

3.9.1 Trust boundaries

1. **Browser ↔ server:** everything arriving from a browser (REST bodies, Socket.IO payloads, tokens) is untrusted. All checks happen on the server; client-side role checks are only for convenience.
2. **Server ↔ database:** only COMP-08 talks to MongoDB, through Mongoose models with schema-typed query values (no string-built queries); board-object adds and updates use the model's native collection with values already validated by the operation schemas (SEC-REQ-005) and the board ID converted to an ObjectId, which avoids re-casting large point arrays; the database listens only on the server host.

3.9.2 Authentication (same tokens for REST and Socket.IO)

- **Registered users:** `POST /api/v1/auth/login` verifies the bcrypt hash (cost factor 12, D13a) and returns a signed JWT (HS256; secret from an environment variable, never committed) with claims `sub` (user ID), `jti` (token ID), `typ: "session"` and `exp` = issue time + 24 hours (D8c). Logout stores the `jti` in `revokedTokens` until expiry (SEC-REQ-002).
- **Guests:** `POST /api/v1/links/resolve` validates a share-link token and returns a guest JWT with claims `guestId`, `boardId`, `linkId`, `role`, `typ: "guest"` and `exp` = min(link expiry, issue time + 24 hours). A guest token is accepted only for its own

board; it is kept in the browser's `sessionStorage`, so it ends with the browser session (FR-046). When a guest token expires while the link is still valid, the client returns the guest to the guest entry page, where the link can be resolved again for a new token.

- **Transport of tokens:** always in the `Authorization: Bearer` header (REST) or the Socket.IO handshake `auth` field — never in cookies or URLs. Because no cookie carries authority, cross-site request forgery does not apply; CORS on Express and Socket.IO allows only the configured client origin.
- **Verification:** COMP-04 `verifyToken` checks signature, `exp`, `typ` and (for sessions) the `jti` revocation list. COMP-03 calls it for every protected route; COMP-06 calls it in connection middleware before a socket can join a room, and re-checks expiry on every `op:submit` so an expired token cannot keep editing over an open socket. Failures → `401 UNAUTHENTICATED` / `connect_error` with code `UNAUTHENTICATED`.

3.9.3 Authorisation (same policy for REST and Socket.IO)

- COMP-05 provides one function `authorize(actor, boardId, action)` implementing the FR-051 matrix. The role is always loaded from server data: `boards.ownerId`, `memberships`, or — for a guest — the `shareLinks` entry of the token's `linkId`, re-checked for expiry and revocation. A role or board ID inside a request payload is never trusted (SEC-REQ-003). Besides the FR-051 actions, the policy has an Owner-only `rename` action for the board-rename endpoint (issue #13; outside the SRS baseline, SRS §2.7).
- Every REST route and every Socket.IO event handler calls `authorize` before reading or changing board data. A requester with no role on the board receives `BOARD_NOT_FOUND` (existence not revealed); a requester whose role is too low receives `FORBIDDEN`. Rejected attempts change nothing, are not broadcast and are logged as `authz.denied` (SEC-REQ-008).
- Role changes, member removal, ownership transfer, link revocation or expiry and board deletion are pushed to COMP-06, which updates the cached role of affected sockets immediately and disconnects sockets that lost access within 5 seconds (FR-047 to FR-050, D3a).

3.9.4 Share-link security

- Token = 32 random bytes from Node.js `crypto.randomBytes` (256 bits; meets SEC-REQ-004's 128-bit minimum), encoded base64url (43 characters, D8e). Link URL: `https://<host>/join#<token>` — the token is in the URL fragment, so browsers do not send it in HTTP request lines and it does not appear in server access logs; the client posts it to `/links/resolve`.
- Only `SHA-256(token)` is stored in `shareLinks.tokenHash`; the plain token is shown to the Owner once at creation.
- Expiry (`expiresAt`) and revocation (`revokedAt`) are checked on resolve and every time a guest's role is evaluated. A COMP-06 timer (every 1 s) disconnects guest sockets whose link has expired, which keeps disconnection within the 5 s of FR-048.

3.9.5 Threat model (STRIDE)

STRIDE category	Threat to this system	Affected components	Mitigation	Requirement / test
Spoofing	Attacker reuses a stolen session token, guesses a share-link token, or brute-forces a password.	COMP-03, COMP-04, COMP-06	24-hour token expiry and logout revocation; guest tokens bound to one board; 256-bit link tokens; login rate limit; TLS in deployment.	SEC-REQ-002, SEC-REQ-004, SEC-REQ-006, SEC-REQ-007; TC-104, TC-106, TC-108, TC-109
Tampering	Viewer or guest sends crafted `op:submit` events or changes `boardId/role` fields in payloads; malformed objects corrupt board state.	COMP-05, COMP-06	`authorize` on every event using server-held roles; schema validation and 65,536-byte message limit; whole-object validation before applying.	SEC-REQ-003, SEC-REQ-005; TC-101, TC-107
Repudiation	A user denies having restored a version, revoked a link, transferred ownership or deleted a board.	COMP-09, COMP-05, COMP-07	Security event log (§4.4) with event type, UTC time, request ID, actor (`null` before identification), board (`null` if none) and reason code; `createdBy` on versions and links.	SEC-REQ-008; TC-104
Information disclosure	Leaked share link; non-member reads a board by guessing its ID (IDOR); tokens or passwords in logs; user text rendered as HTML (XSS) steals tokens.	COMP-01, COMP-03, COMP-05, COMP-09	Link expiry and revocation; `authorize` on every read with 404 for non-members; token in URL fragment and hashed at rest; log redaction; text rendered as plain text only.	SEC-REQ-003, SEC-REQ-004, SEC-REQ-005, SEC-REQ-008; TC-102, TC-103, TC-104, TC-107
Denial of service	One client floods operations or sends oversized payloads; a board grows without bound; brute-force login.	COMP-03, COMP-06, COMP-08	30 operations/s per connection; 64 KiB (65,536-byte) message limit (`maxHttpBufferSize`, measure M1); 15 MiB board content limit; login rate limit; capacity verified by TC-019 (Pass on 12-10-2026: 10 clients, p95 80 ms, 0 rejected).	SEC-REQ-005, SEC-REQ-006, FR-008, NFR-002; TC-107, TC-108, TC-022, TC-019
Elevation of privilege	Editor attempts Owner actions (clear, restore, manage links, transfer ownership); guest calls Owner endpoints; removed member keeps an open socket.	COMP-03, COMP-05, COMP-06	Permission matrix in one place (COMP-05); Owner actions checked on the server; guest tokens can never act as Owner; role cache updated and socket disconnected on removal or revocation.	FR-051, SEC-REQ-003, FR-047, FR-049, FR-050; TC-101, TC-102, TC-103, TC-018, TC-023

3.9.6 Security control mapping

Security requirement	Design control(s)	Component(s)	Validating test(s)
SEC-REQ-001	Length check 8–72 in schema; bcrypt (cost 12) with per-password salt; password fields redacted from logs	COMP-03, COMP-04, COMP-08, COMP-09	TC-105

Security requirement	Design control(s)	Component(s)	Validating test(s)
SEC-REQ-002	JWT session tokens (24 h, `jti` revocation on logout); board-bound guest tokens (min(link expiry, 24 h)); verification on every route, socket connection and operation	COMP-03, COMP-04, COMP-06	TC-106
SEC-REQ-003	Single `authorize` policy with server-held roles; 404/403 rule; no trust in payload roles or board IDs	COMP-03, COMP-05, COMP-06	TC-101, TC-102
SEC-REQ-004	256-bit CSPRNG tokens; SHA-256 hash at rest; token only in URL fragment; expiry and revocation checks; 1 s expiry timer	COMP-05, COMP-06, COMP-08	TC-103, TC-104
SEC-REQ-005	JSON-schema validation of every REST body and Socket.IO payload; 65,536-byte `maxHttpBufferSize` (M1); text rendered with React text nodes only (no `dangerouslySetInnerHTML`)	COMP-01, COMP-03, COMP-06	TC-107
SEC-REQ-006	Login limiter (5 failures / entered e-mail address / 15 min, registered or not, keyed by an HMAC of the address; identical status, code and message either way, §4.4); token-bucket limiter (30 ops/s / connection)	COMP-03, COMP-06	TC-108
SEC-REQ-007	TLS 1.2+ at the deployment's HTTPS endpoint; HTTP→HTTPS redirect; `wss://` for Socket.IO	COMP-03, COMP-06 (deployment configuration)	TC-109
SEC-REQ-008	Security event record (§4.4) created by the detecting component (COMP-03 to COMP-07) and written by COMP-09 with a field allow-list: event, timestamp, request ID, actor or `null`, board or `null`, affected account (`subjectId`) or `null`, outcome and reason; passwords, tokens, `Authorization` headers, entered e-mail addresses and board content are never written	COMP-03, COMP-04, COMP-05, COMP-06, COMP-07, COMP-09	TC-104

Deployment assumptions for SEC-REQ-007 (as implemented in `server/src/config.js`, `rest/app.js` and `server.js`). TLS ends at a reverse proxy in front of the server; the server itself speaks plain HTTP. `FORCE_HTTPS=true` enables the controls below and is accepted only together with `TRUST_PROXY=true` and an `https://` `PUBLIC_URL`, otherwise the server refuses to start (without trusted `X-Forwarded-Proto` every request would look like HTTP and be redirected in a loop). `TRUST_PROXY` trusts exactly one proxy hop. With `FORCE_HTTPS`: a plain-HTTP request receives a 308 redirect to the origin of `PUBLIC_URL` (never to the client-supplied `Host` header); every response of the Express application (REST API and built client) carries `Strict-Transport-Security: max-age=31536000`; a Socket.IO connection is refused unless the proxy reports HTTPS; and the built client's Content-Security-Policy allows `connect-src 'self' wss:` only (without `FORCE_HTTPS`, local development, also `ws:`). The proxy must therefore *set* (overwrite, not append) `X-Forwarded-Proto`, forward WebSocket upgrades, and offer TLS 1.2 or later; the server must listen only on the loopback interface (`HOST=127.0.0.1`, the default) or a private network, because a client that reaches it directly could claim `X-Forwarded-Proto: https`. These proxy and network settings are deployment configuration and are verified only by TC-109.

3.9.7 Secrets handling

The JWT signing secret and database connection string are read from environment variables (`.env` locally, excluded from Git). No secret, password or token appears in the repository, in documentation examples (placeholders such as `<token>` are used) or in logs. Test builds use a separate test secret (`.env.test`, also excluded from Git).

4. Design

4.1 Design Overview

This section is a design specification (historical v1.0 note: written before any application code existed). Implementation status and test evidence: execution of 12-10-2026 (uncommitted working tree on base commit 82af885; see `docs/IMPLEMENTATION_STATUS.md`).

4.1.1 Board object model and operations

Every board object has a client-generated UUID `id`, a `kind` (`stroke`, `rect`, `ellipse`, `line`, `text`) and kind-specific properties: `points` (array of `[x, y]`, at most 2,000 points) for strokes; `x`, `y`, `w`, `h` for rectangles and ellipses; `x1`, `y1`, `x2`, `y2` for lines; `x`, `y`, `text` (≤ 500 characters) for text; and `color` (`#RRGGBB`) and `strokeWidth` (1–20) for all kinds.

Four operation types travel over API-01: `object:add` (full object), `object:update` (object `id` plus the **complete new values** of the changed properties), `object:delete` (`id`) and `board:clear` (Owner only). Moving, restyling and editing text are `object:update`; erasing sends one `object:delete` per touched object.

4.1.2 Server processing of an operation (COMP-06)

Operations for one board pass through a **per-board FIFO queue** and are processed one at a time, so the server's acceptance order is a total order. For each `op:submit` the server:

1. Checks the per-connection rate limit (SEC-REQ-006) and the payload schema and size (SEC-REQ-005).
2. Calls `authorize(actor, boardId, action)` with the role held on the server (SEC-REQ-003).
3. Checks preconditions: update or delete of a missing object → `OBJECT_NOT_FOUND`; `contentBytes` (conservative BSON size, §3.6) above 15 MiB after the change → `BOARD_FULL` (FR-008), checked before any write; `board:clear` on an empty board → accepted as a no-op without a sequence number (FR-007).
4. Assigns `seq = board.seq + 1` and writes the effect to `boardObjects` (with `updatedSeq = seq`) and the new `seq` to `boards`, both with journaled write concern (`{w: 1, j: true}`). A `board:clear` does not use steps 4–5: it follows the journalled clear procedure of §3.6 (before-clear version, `pendingOp` commit, deletion, completion), so a clear is never acknowledged or broadcast before it is complete and an interrupted clear is completed at start-up (FR-007).

5. Only after both writes succeed: updates the in-memory board cache, sends the acknowledgement to the submitter and broadcasts `op:applied` to the other sockets in the room (**persist before acknowledge and broadcast**, FR-027). If a write fails, the operation is rejected with `INTERNAL`; nothing is acknowledged or broadcast and `seq` is not advanced.

Crash safety without multi-document transactions (a standalone MongoDB does not support them): on start-up COMP-06 sets each board's `seq` to the maximum of the stored `boards.seq` and the largest `updatedSeq` in `boardObjects` — as step 2 of the start-up recovery order in §3.6, after step 1 has completed every journalled clear or restore (`boards.pendingOp`). A sequence number can only be reused if the server crashed after writing but before acknowledging, in which case no client ever saw it.

4.1.3 Acknowledgement and client reconciliation (COMP-02)

Acknowledgement contents. Success: `{ok: true, clientOpId, seq}`. Rejection: `{ok: false, clientOpId, error: {code, message, requestId}}`. The server never alters an accepted payload, so the acknowledged effect equals the submitted operation.

Client state. Each client keeps:

- `confirmed` — the canonical board state up to `lastSeq` (all operations with sequence number ≤ `lastSeq` applied in order);
- `pending` — its own submitted but unacknowledged operations, in submission order;
- `buffer` — events received with a sequence number greater than `lastSeq + 1`.

The **rendered view** is `confirmed` with the `pending` operations applied on top (optimistic display).

Processing rules.

1. Every accepted operation reaches a client exactly once with its sequence number: as `op:applied` (operations of others) or as a success acknowledgement (own operation).
2. An event with `seq = lastSeq + 1` is applied to `confirmed`, `lastSeq` is incremented, and any buffered events that are now in order are applied. For a success acknowledgement the operation is also removed from `pending`.
3. An event with `seq ≤ lastSeq` is a duplicate or already contained in a resynchronised state: it is not applied again (an acknowledgement still removes its operation from `pending`).
4. An event with `seq > lastSeq + 1` is placed in `buffer`. A **gap** exists while `buffer` is non-empty. If the gap is not closed within 1 s, or `buffer` holds more than 100 events (D6b), the client resynchronises: it emits `board:join`, replaces `confirmed` with the returned state, sets `lastSeq` to the returned `seq`, discards `buffer` and keeps `pending`.
5. On a rejection acknowledgement the operation is removed from `pending`, the view is recomputed (its local effect disappears) and the error message is shown (FR-026). Later

pending operations that depended on it (e.g. an update of a rejected new object) are rejected by the server with `OBJECT_NOT_FOUND` and handled the same way.

6. If no acknowledgement arrives within 10 s (D6b), the client treats the connection as unhealthy, disconnects and follows the reconnection procedure.

Convergence. Each client applies exactly the server's accepted operations in sequence order to `confirmed`. Because updates carry complete property values, applying the same sequence to the same starting state gives the same result on every client and on the server, so all `confirmed` states equal the server state at the same `seq`. Pending operations only affect the submitter's view until they are acknowledged (then part of `confirmed`) or rejected (then removed). Since Socket.IO delivers messages on one connection in order and the server emits in sequence order, rules 3 and 4 are safeguards rather than the normal path.

Conflict policy (last write wins). When two updates change the same property of the same object, the one with the higher sequence number is applied last and determines the final value on every client (FR-023). Different properties or different objects are both preserved (FR-022). A delete followed by an update of the same object rejects the update with `OBJECT_NOT_FOUND`. Limitation: a concurrent edit of the same text object keeps only the later text; no merge is performed.

4.1.4 Reconnection (COMP-02, COMP-06)

- The Socket.IO client reconnects automatically with `reconnectionDelay` 500 ms, `reconnectionDelayMax` 2,000 ms and `randomizationFactor` 0.2 (D6b). The library default maximum of 5,000 ms would make NFR-004 (5 s) unattainable, so the lower maximum is required.
- The server detects a silently lost connection with the Socket.IO heartbeat: `pingInterval` 10 s and `pingTimeout` 10 s (library defaults 25 s and 20 s; `server/src/server.js`), so the disconnected indicator of FR-025 appears within about 20 s of an outage (TC-009). The heartbeat is answered on the event loop, so a connection is also dropped if either side's event loop is blocked for longer than this; the longest synchronous server step is saving a full-board version (BSON size check and serialisation of up to about 15 MiB; measured 0.5–0.8 s in plain Node.js on the development laptop, 15-10-2026). In TC-022, which runs the server and both test clients in one Jest process, the same step took several seconds and, under artificial CPU load, longer than the heartbeat, so the test clients reconnected (STP TC-022).
- While disconnected the client shows the indicator and disables tools (FR-025).
- After `connect` the client emits `board:join`, replaces `confirmed` with the returned state and `lastSeq` with the returned `seq`, and clears `buffer`. Pending operations whose acknowledgement was lost cannot be distinguished from operations that never reached the server; version 1.0 therefore **discards all pending operations** after resynchronisation and shows the "unconfirmed changes may not have been saved" notice if any were pending (FR-025). Idempotent resubmission using `clientOpId` is future work (§4.6).

- The client records the `board-ready` performance mark after the resynchronised state is rendered and tools are enabled (used by NFR-004).

4.1.5 Version triggers (COMP-07)

COMP-07 keeps one timer per active board with interval *T* (configuration `VERSION_INTERVAL_MS`; 5 minutes, D11a). It also receives "before clear" and "last participant left" events from COMP-06. Every automatic trigger applies the no-change rule of SRS §4.3: COMP-07 compares the board's `seq` with the `seq` recorded in the latest version (`versions.boardSeq`) and creates a snapshot only if they differ. Restore always creates a version. Version creation, clear and restore run as tasks in the board's queue (§3.6), so every snapshot corresponds exactly to one sequence number and no operation is applied while a clear or restore is in progress.

4.1.6 Test hooks (test builds only)

The STP relies on the following hooks (all implemented; historical v1.0 note: none existed when this was written). They are compiled only into test builds (`NODE_ENV=test` / client test build) and are never enabled in a deployed system:

Hook	Purpose	Used by
`performance.mark("board-ready")` in COMP-01	Defines the end point of load and reconnection timing (always present; harmless in production)	NFR-003, NFR-004; TC-009, TC-020
`window.__wbTest.exportState()`	Returns `{lastSeq, objects}` of the client for comparison	TC-006 to TC-008, TC-011
`window.__wbTest.submitRaw(op)`	Submits an operation without client-side validation	TC-010, TC-107
`VERSION_INTERVAL_MS`	Shorter periodic version interval in tests (value recorded in the test log)	TC-012, TC-014
`LINK_EXPIRY_OVERRIDE_SEC`	Allows a 60 s link expiry in tests	TC-103
Test JWT secret in `.env.test`	Lets tests sign deliberately expired tokens	TC-106
`FAULT_INJECT` (`version_write_error`, `after_preclear_version`, `commit_write_error`, `after_pending_clear`, `during_object_delete`, `after_pending_restore`, `after_version_insert`, `during_object_replace`, `after_complete`, `db_error_after_commit`, `recovery_db_error`)	Makes the server stop the process or fail a database write at a chosen point of a clear, a restore or start-up recovery (§3.6)	TC-024, TC-025

4.2 UML Sequence Diagrams

Five project-specific flows are specified (the template requires at least two). Each is split into numbered parts so that its labels stay legible; wide parts are printed on landscape pages. Sources are in `docs/diagrams/src/`.

SD-01 — Live drawing synchronisation, acknowledgement and reconciliation

Realises FR-001, FR-008, FR-020 to FR-023, FR-026, FR-027, FR-051, NFR-001. Editor A adds a stroke: the client shows it optimistically, the server persists it, acknowledges it with

its sequence number and broadcasts it. Then A and Editor B update the same object at nearly the same time and the server's queue order decides the final state. The diagram also shows the gap-repair safeguard, the rollback of a rejected operation and reconnection recovery (protocol rules in §4.1.3–§4.1.4).

Figures 3–8 show the flow in 6 parts.

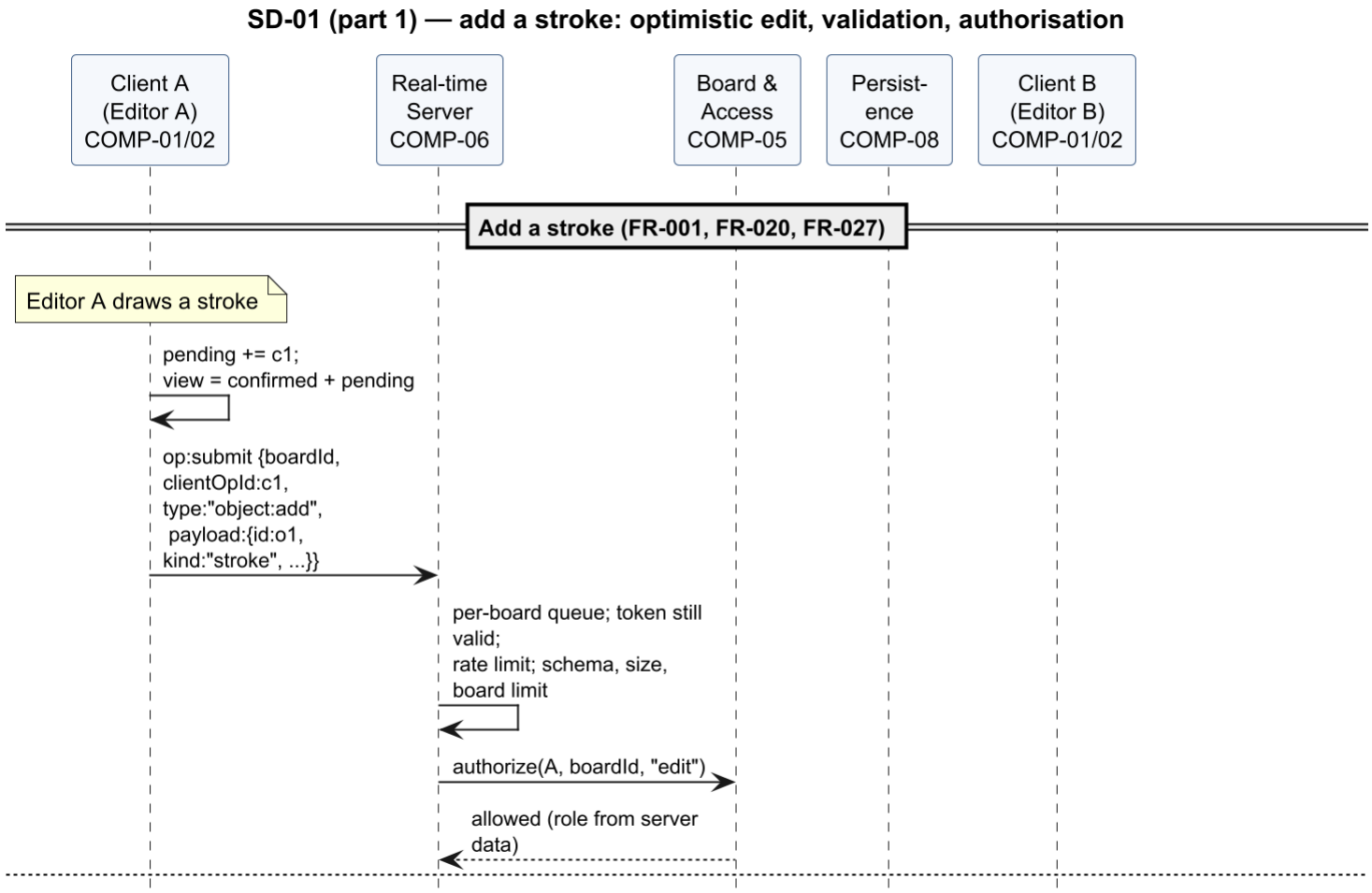


Figure 3 — SD-01 (part 1 of 6): add a stroke: optimistic edit, validation, authorisation

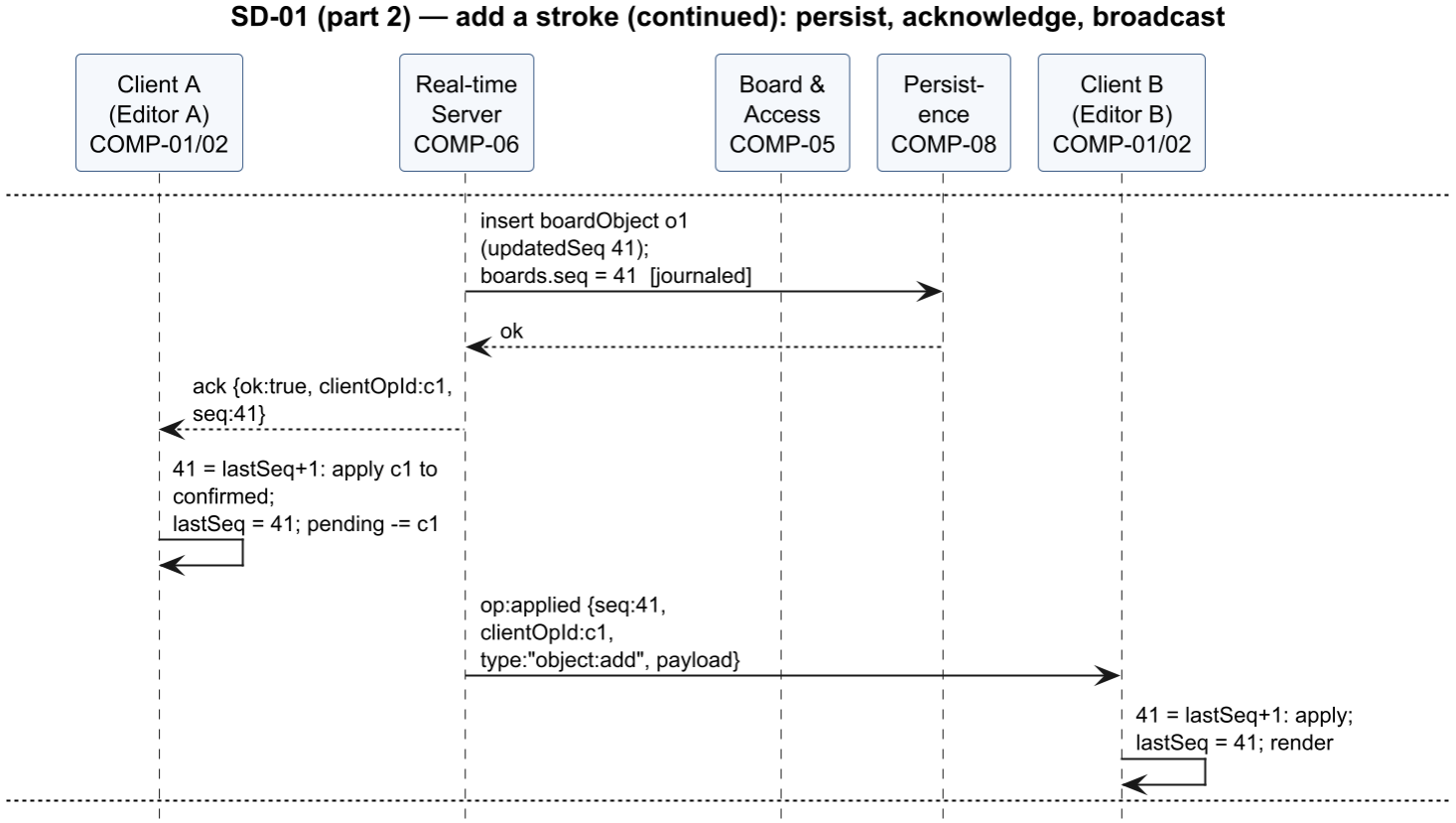


Figure 4 — SD-01 (part 2 of 6): add a stroke (continued): persist, acknowledge, broadcast

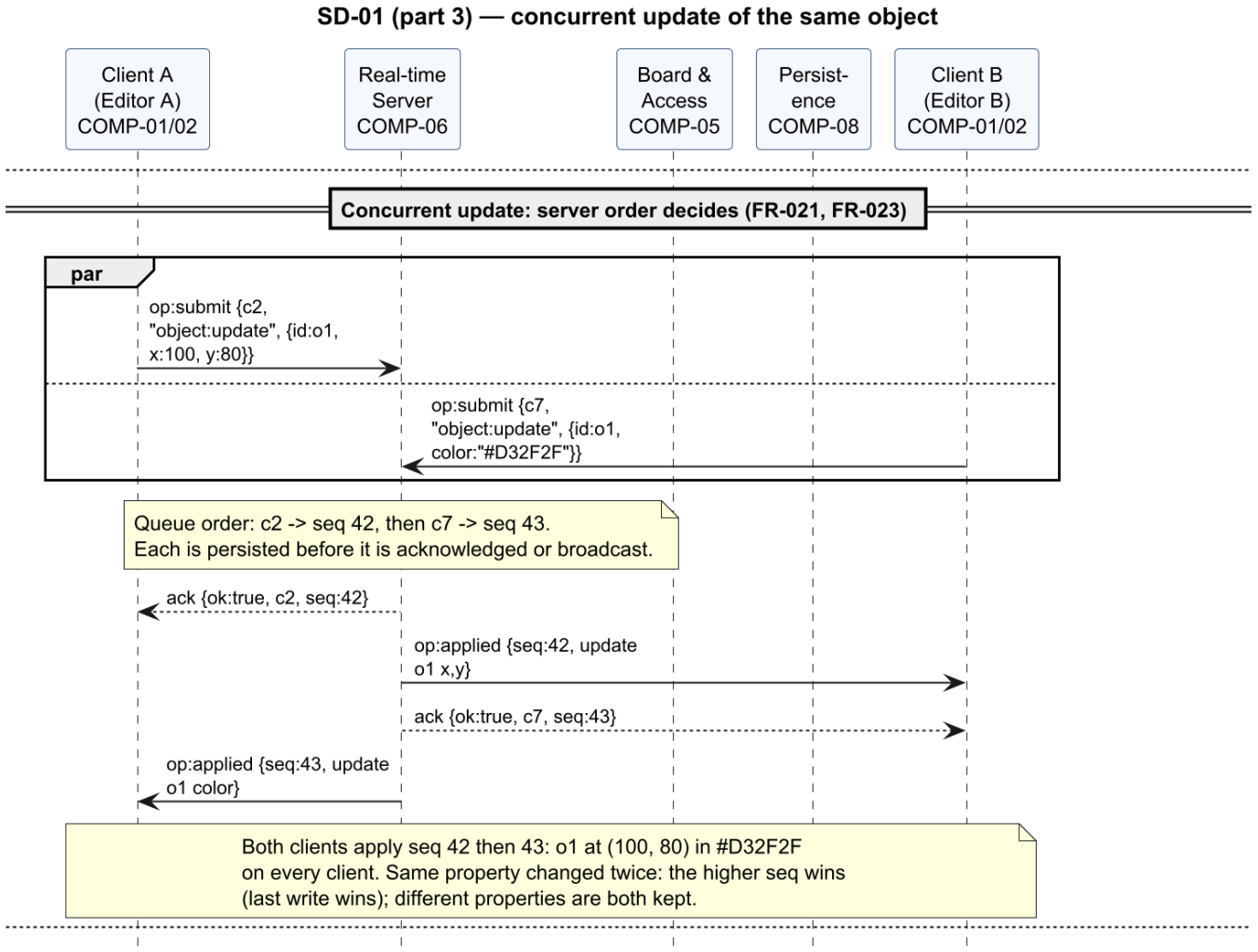


Figure 5 — SD-01 (part 3 of 6): concurrent update of the same object

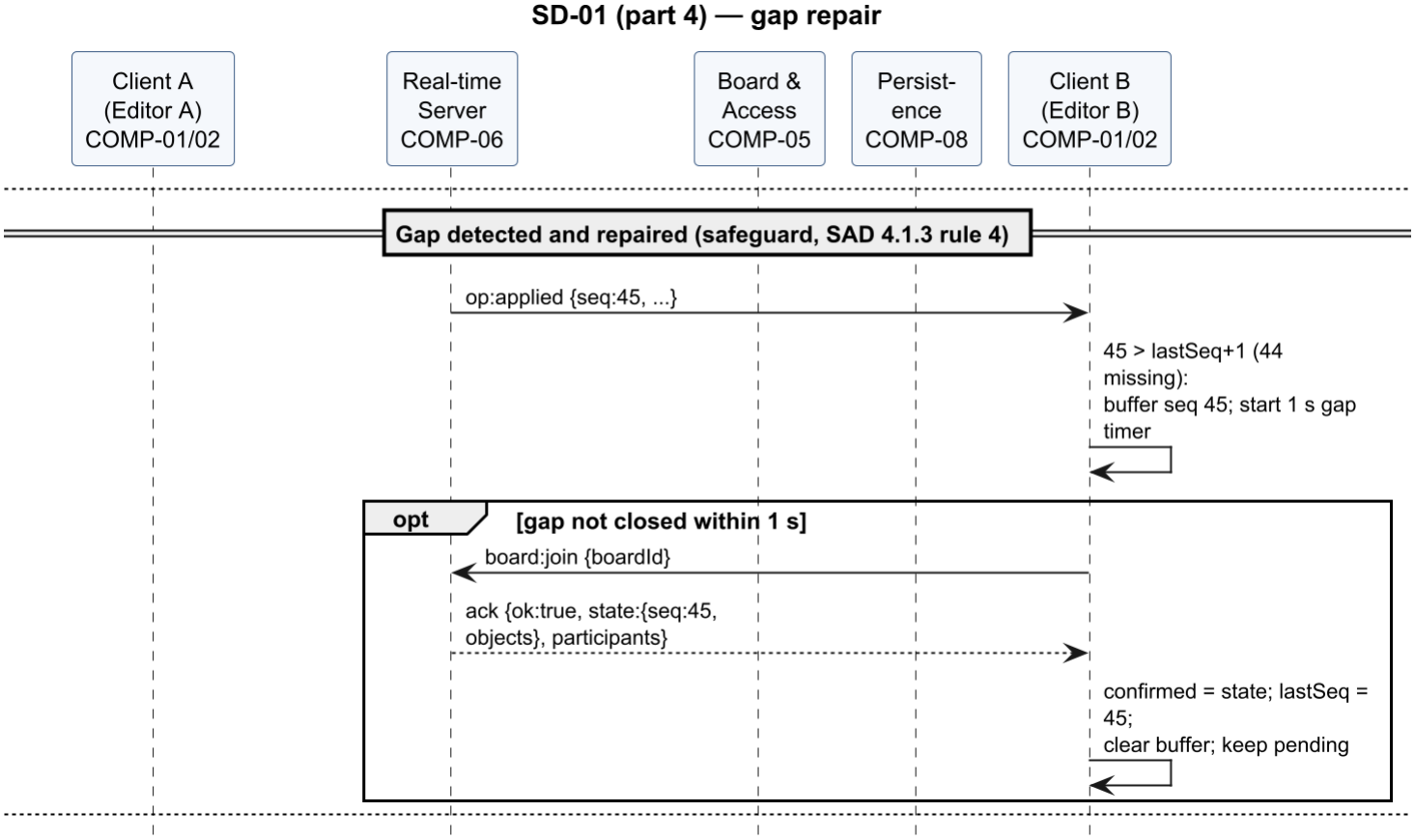


Figure 6 — SD-01 (part 4 of 6): gap repair

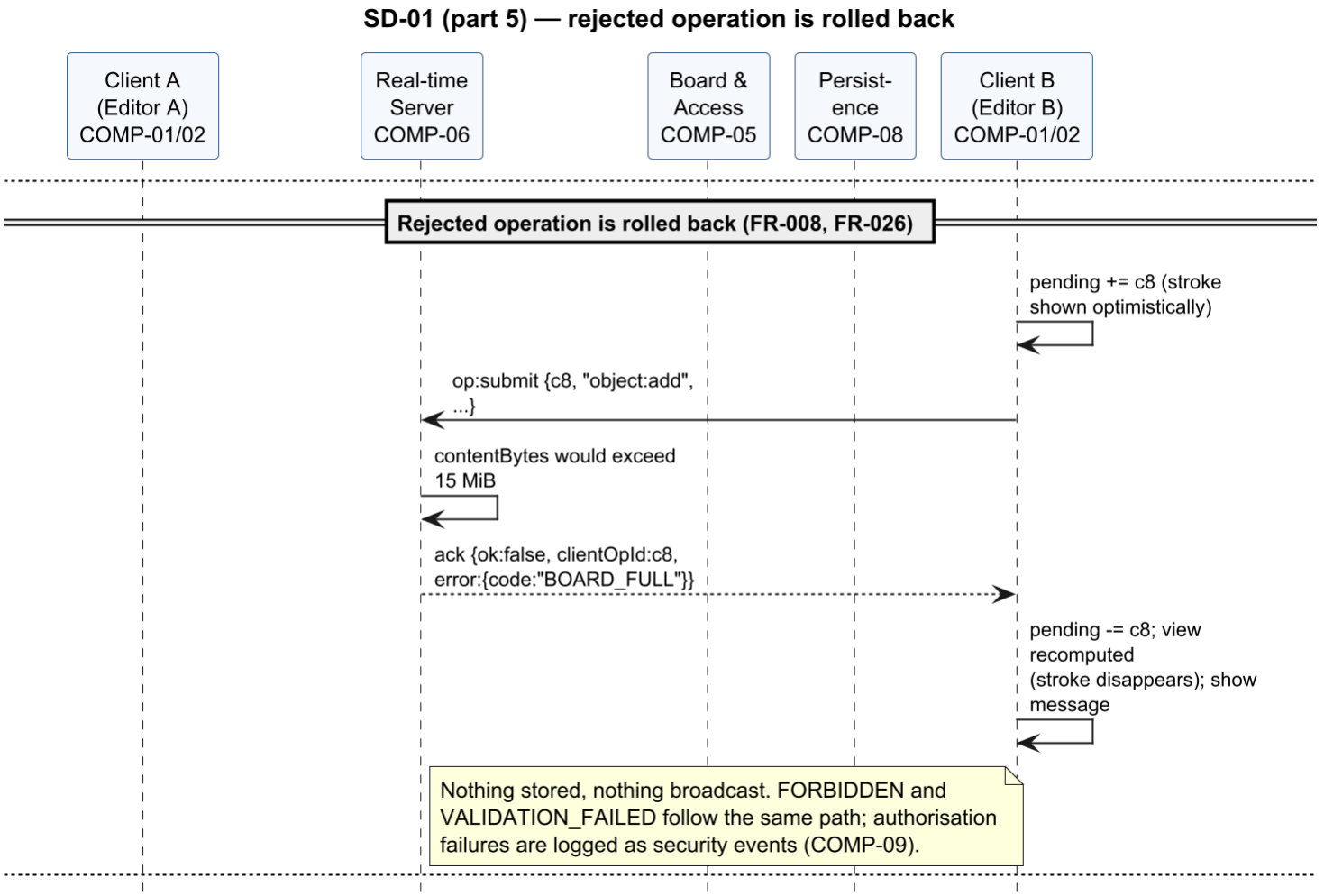


Figure 7 — SD-01 (part 5 of 6): rejected operation is rolled back

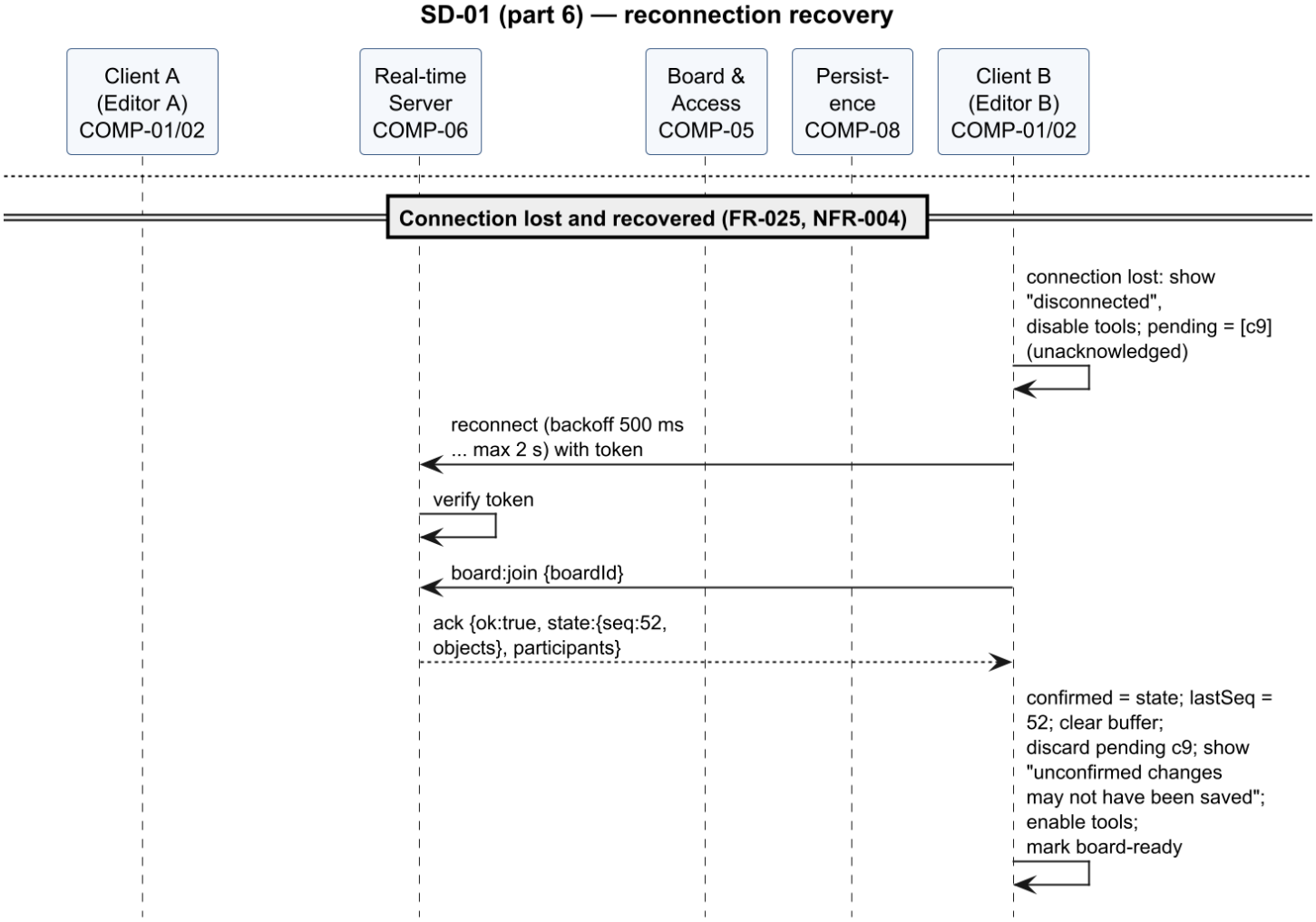


Figure 8 — SD-01 (part 6 of 6): reconnection recovery

SD-02 — Restore a previous version

Realises FR-033, FR-034, FR-051, NFR-005. The Owner restores version *n* through API-03. The restore runs as a task in the board queue: it checks the Owner role and the idempotency key, commits the operation in the durable journal (`boards.pendingOp`), stores the restore version, replaces the board objects, completes the journal entry and only then pushes the restored state to every participant (§3.6). A restore from a non-Owner, or one that fails before its commit, is rejected without any change; a restore interrupted after its commit is completed at start-up.

Figures 9–17 show the flow in 9 parts.

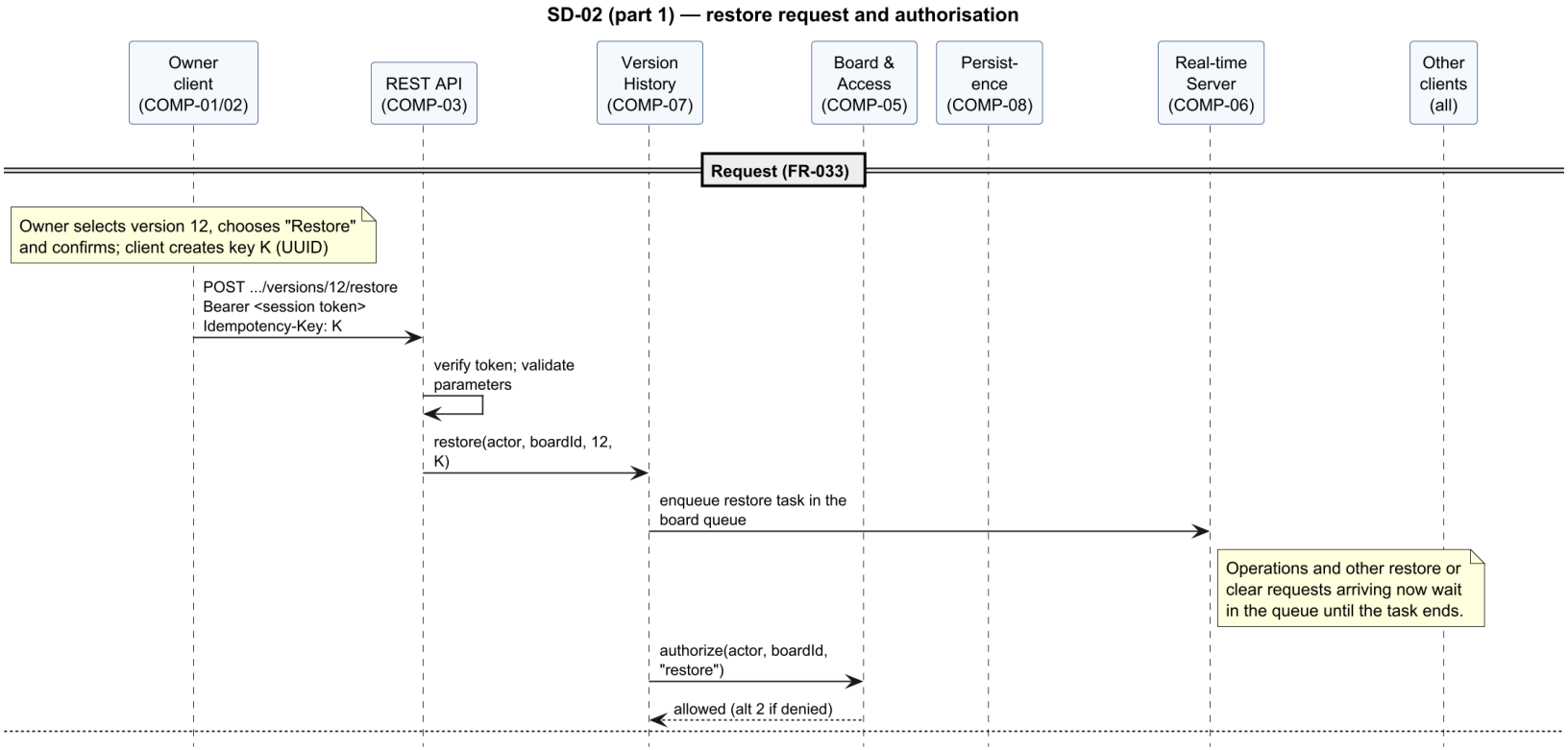


Figure 9 — SD-02 (part 1 of 9): restore request and authorisation

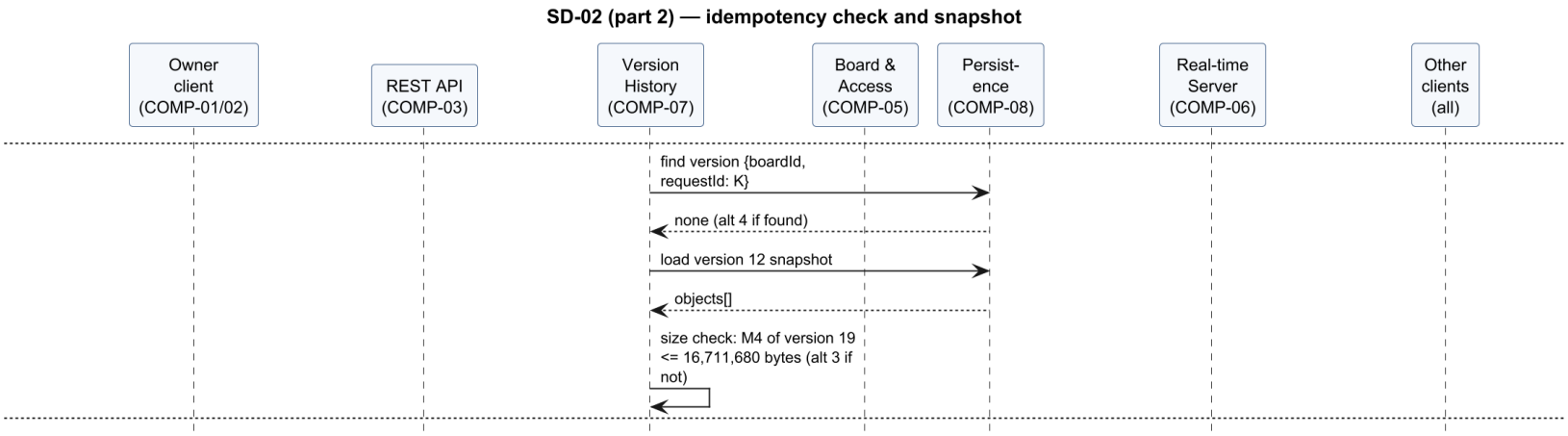


Figure 10 — SD-02 (part 2 of 9): idempotency check and snapshot

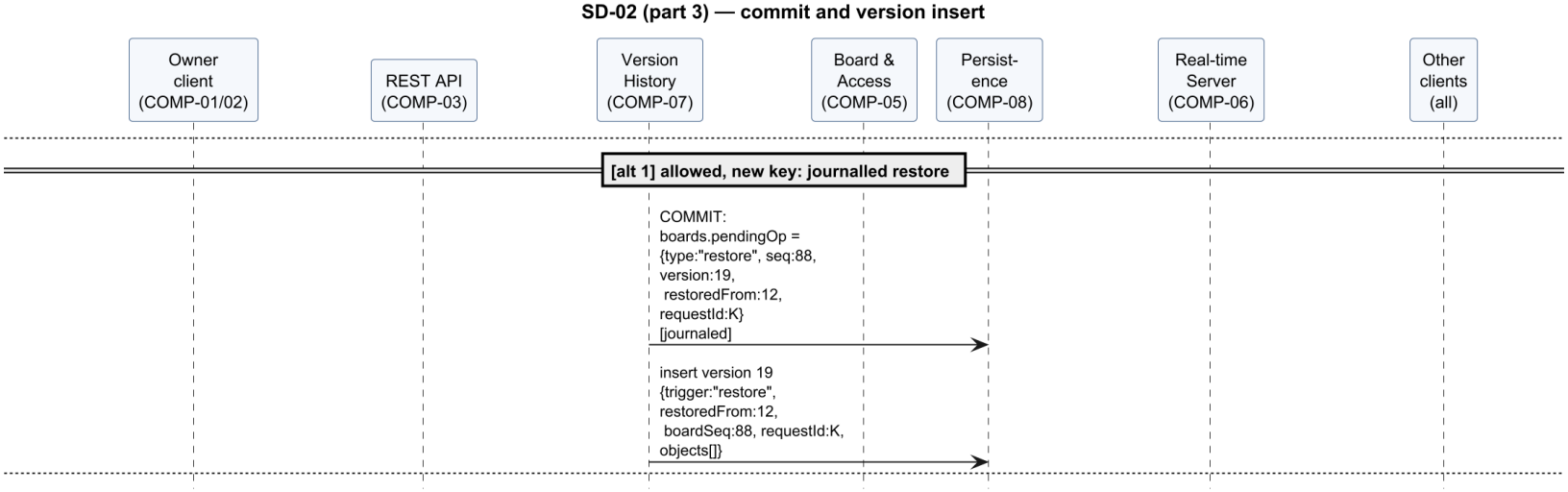


Figure 11 — SD-02 (part 3 of 9): commit and version insert

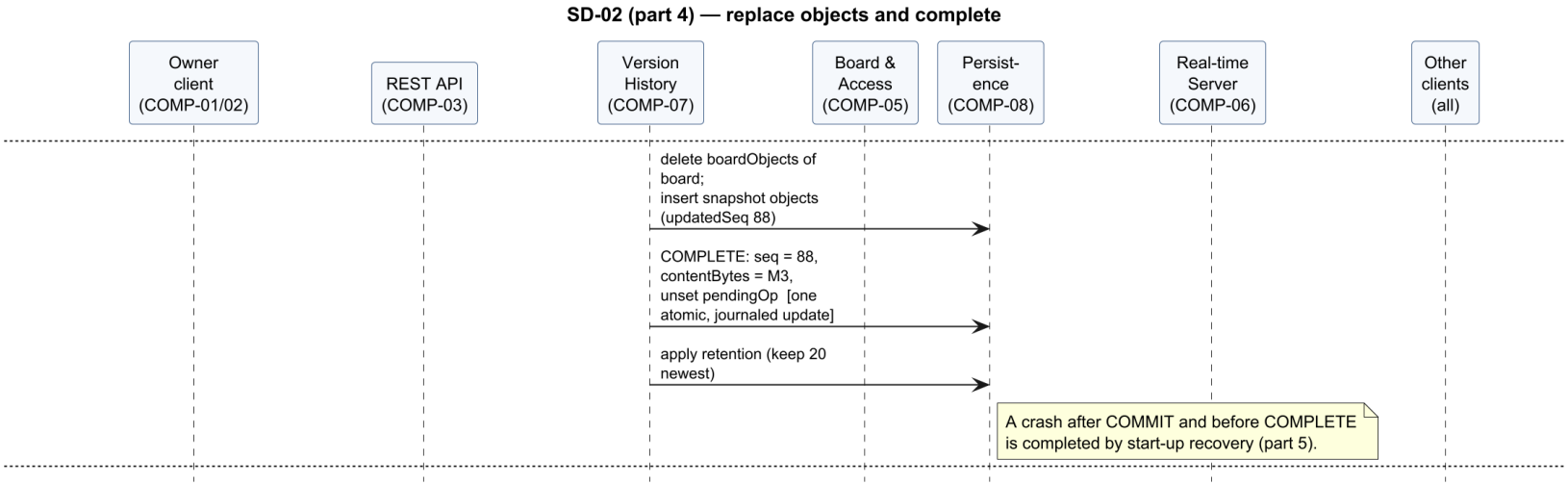


Figure 12 — SD-02 (part 4 of 9): replace objects and complete

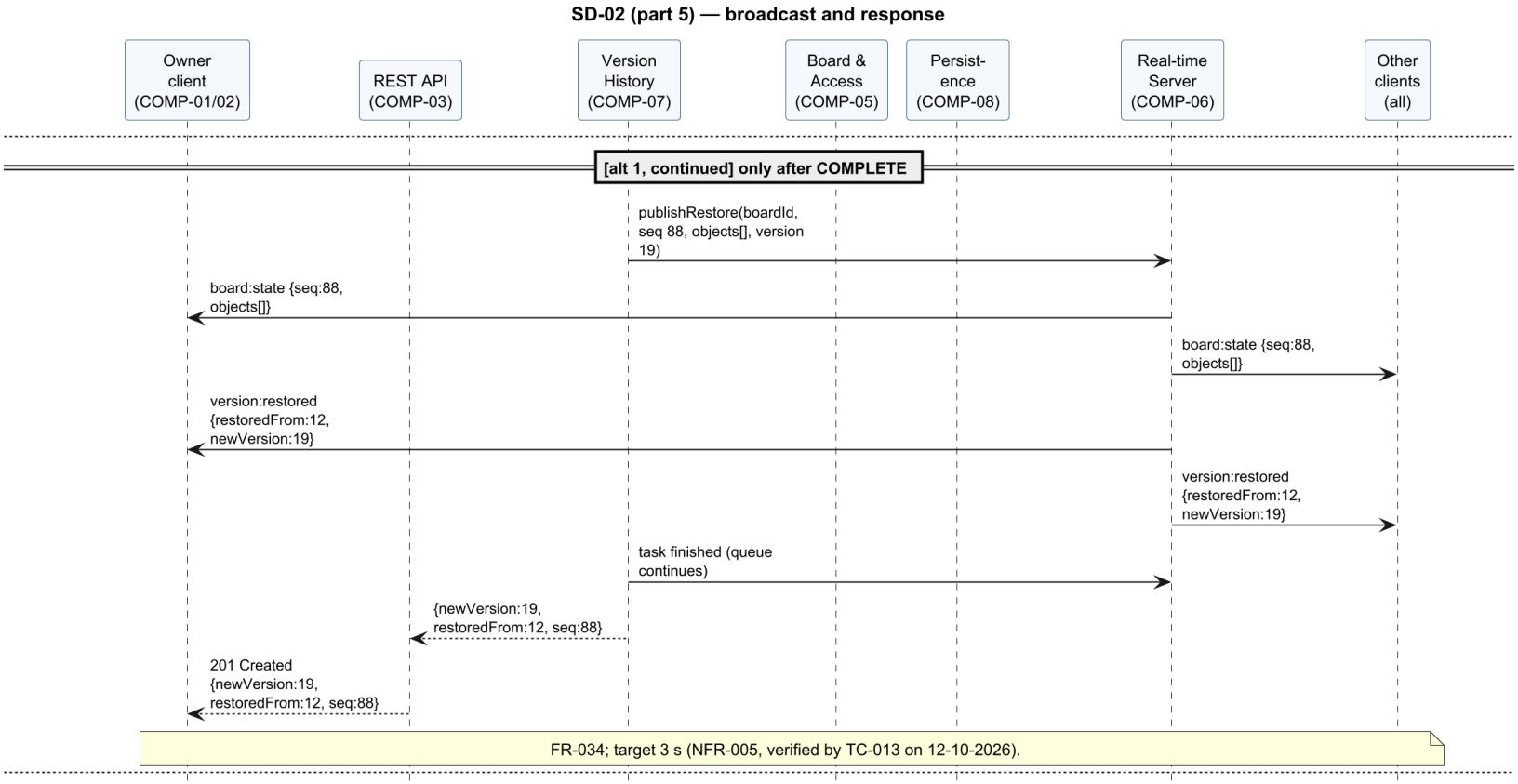


Figure 13 — SD-02 (part 5 of 9): broadcast and response

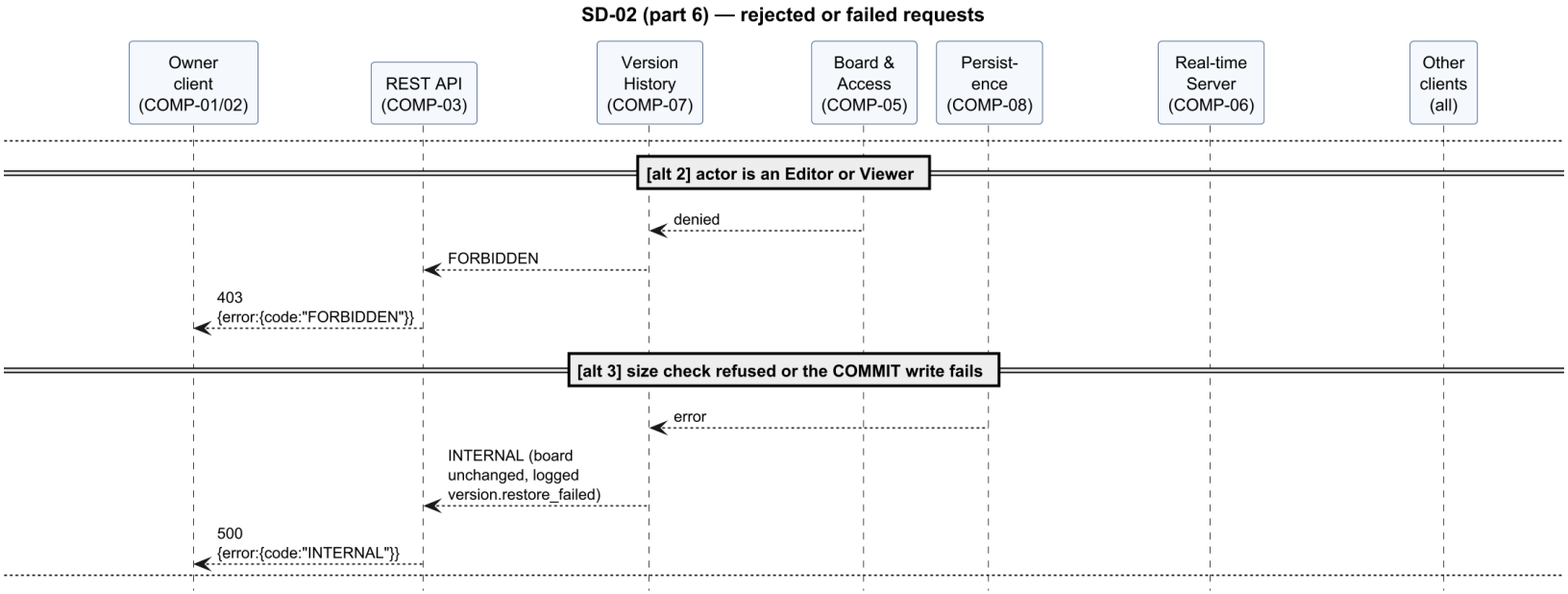


Figure 14 — SD-02 (part 6 of 9): rejected or failed requests

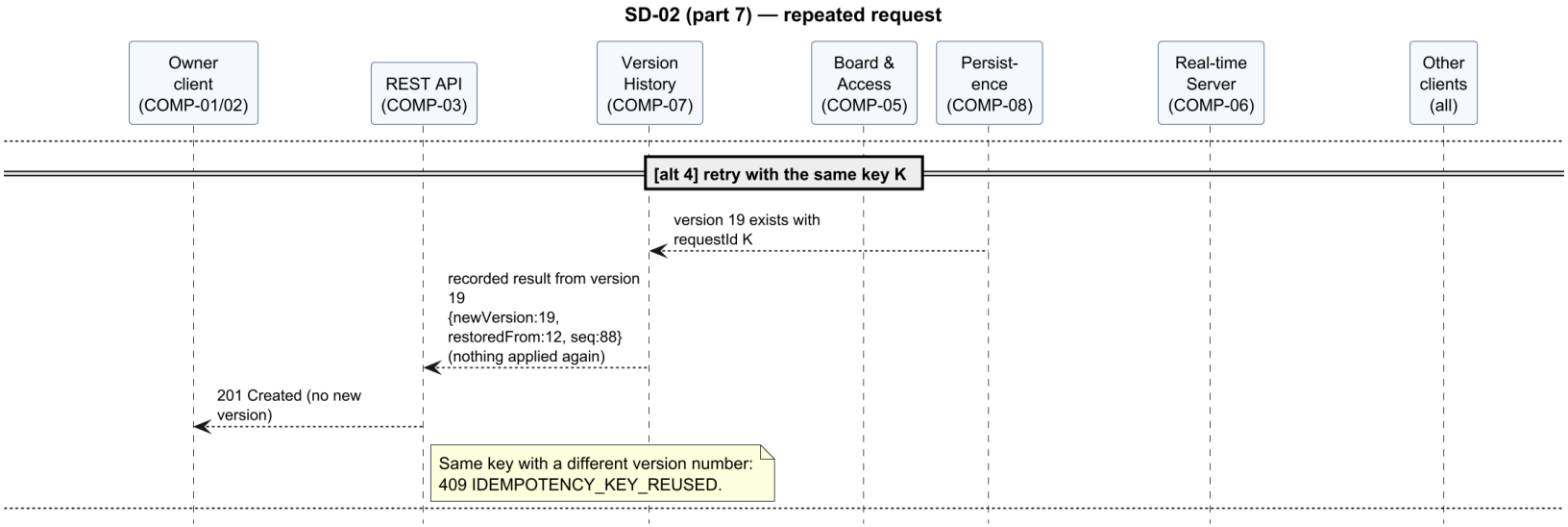


Figure 15 — SD-02 (part 7 of 9): repeated request

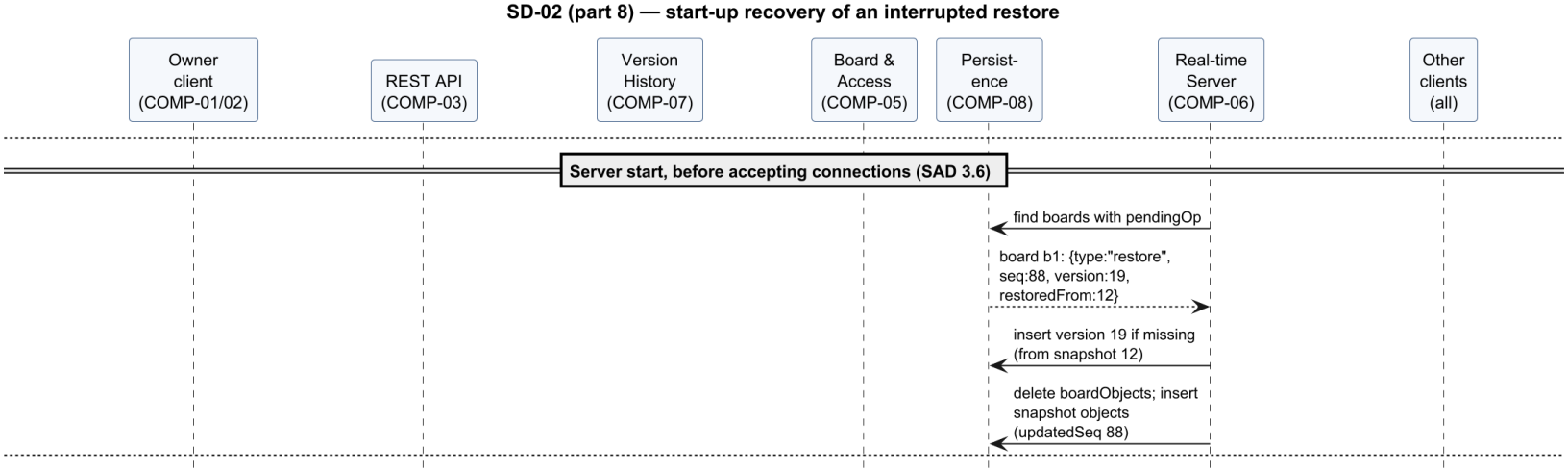


Figure 16 — SD-02 (part 8 of 9): start-up recovery of an interrupted restore

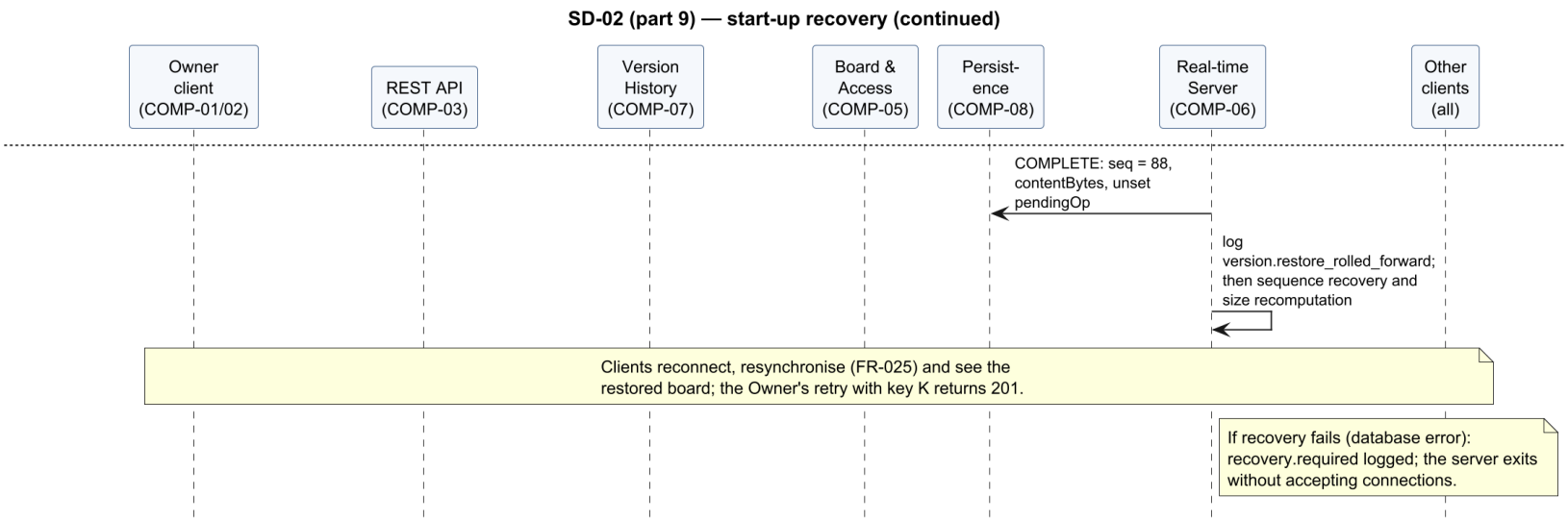


Figure 17 — SD-02 (part 9 of 9): start-up recovery (continued)

SD-03 — Join through a share link, then revocation

Realises FR-024, FR-046, FR-047, FR-048, FR-051, SEC-REQ-002, SEC-REQ-004. A guest opens a share link, the server validates the hashed token and issues a board-scoped guest token, and the guest joins the board room and receives the full state. When the Owner later revokes the link, the guest's socket is notified and disconnected.

Figures 18–22 show the flow in 5 parts.

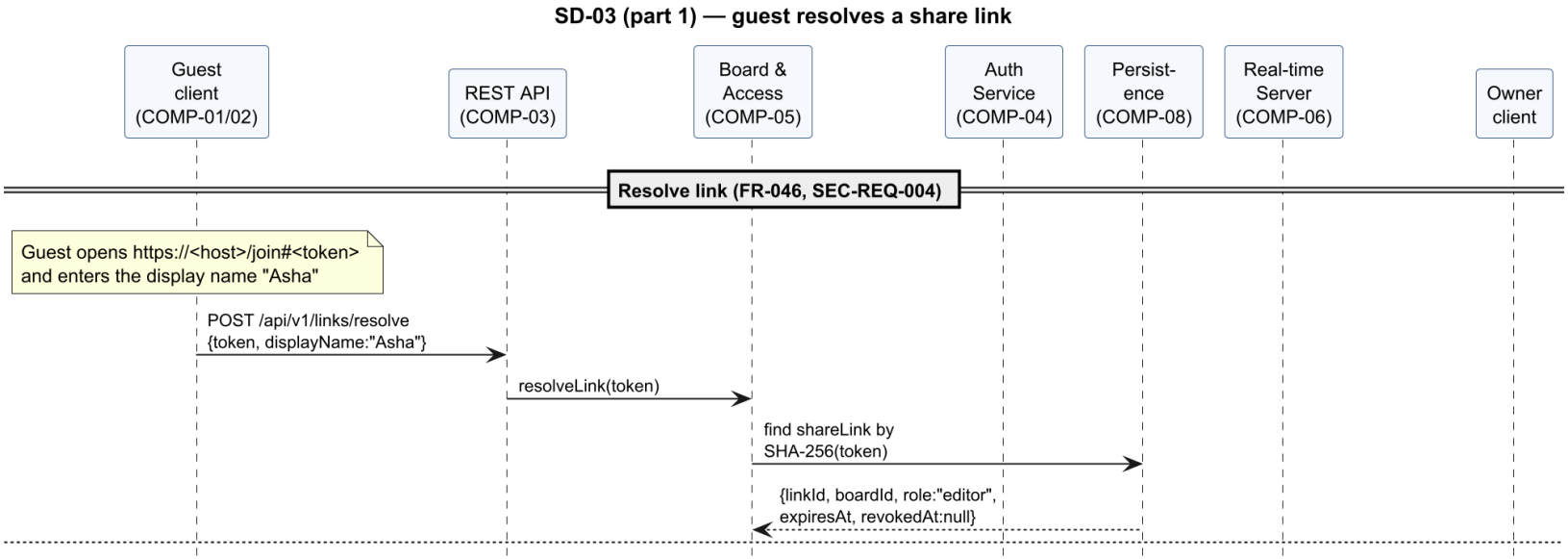


Figure 18 — SD-03 (part 1 of 5): guest resolves a share link

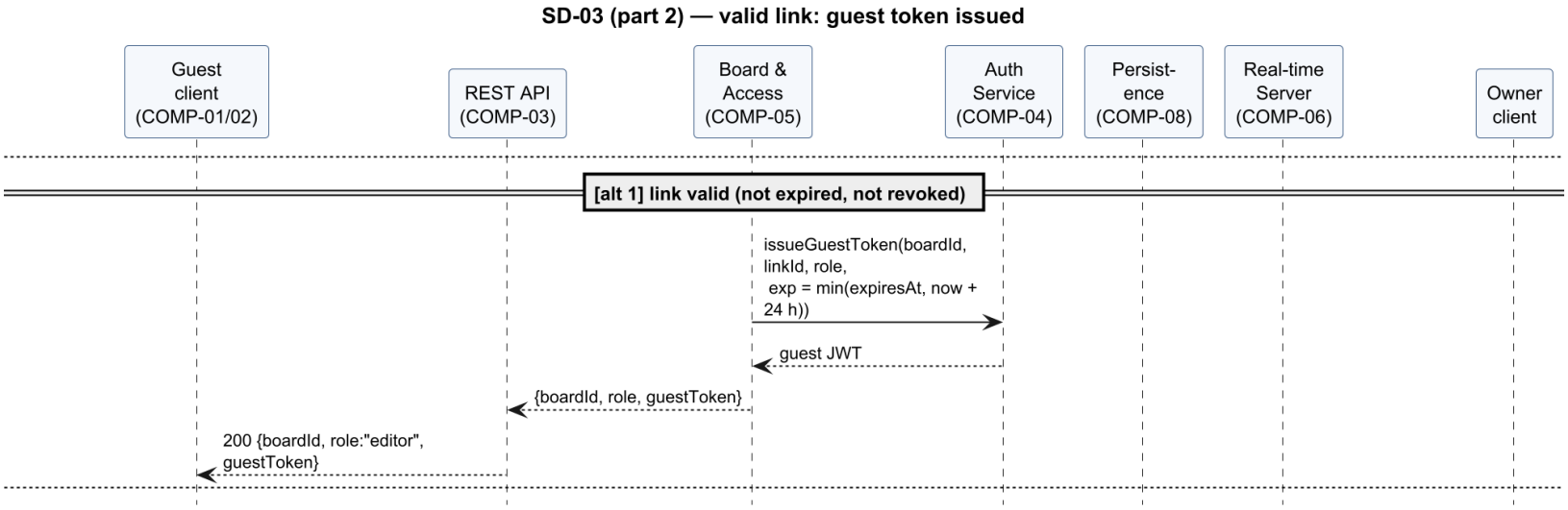


Figure 19 — SD-03 (part 2 of 5): valid link: guest token issued

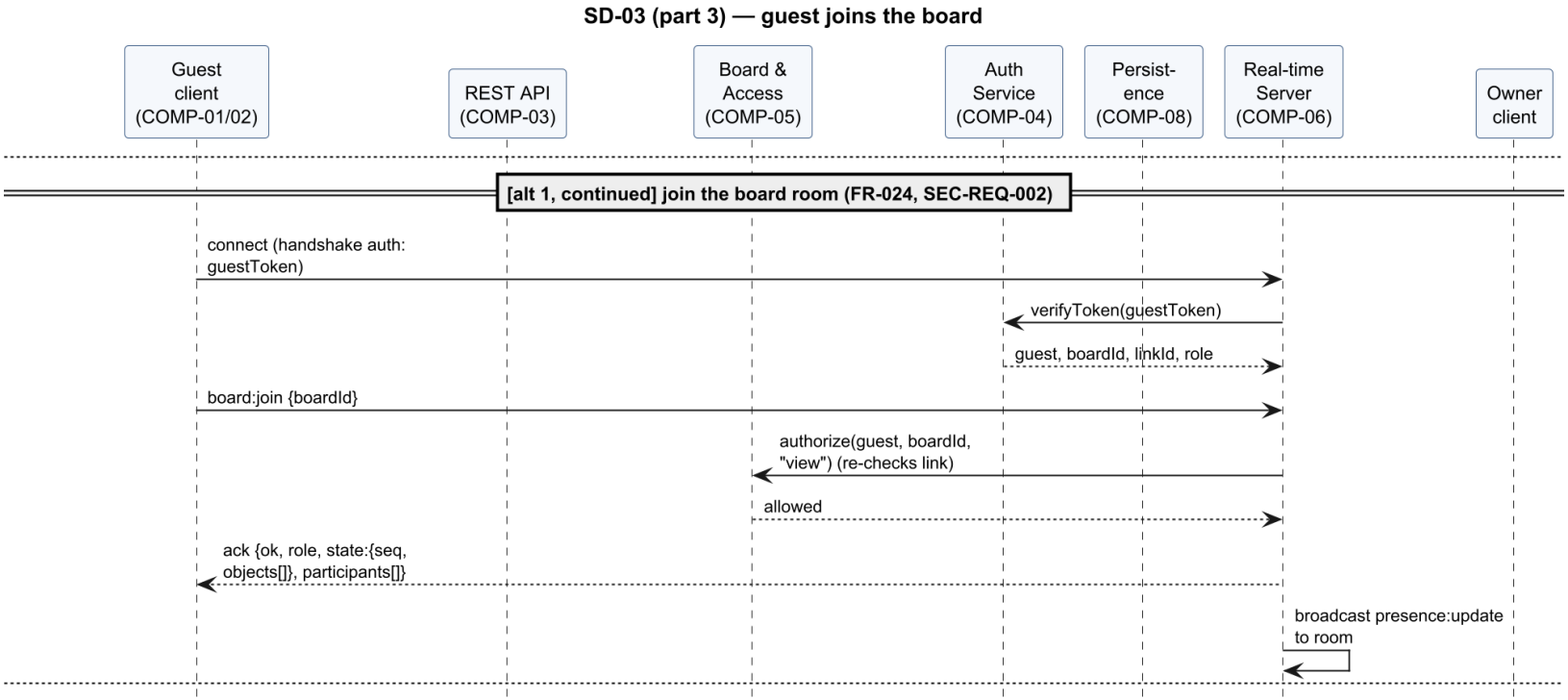


Figure 20 — SD-03 (part 3 of 5): guest joins the board

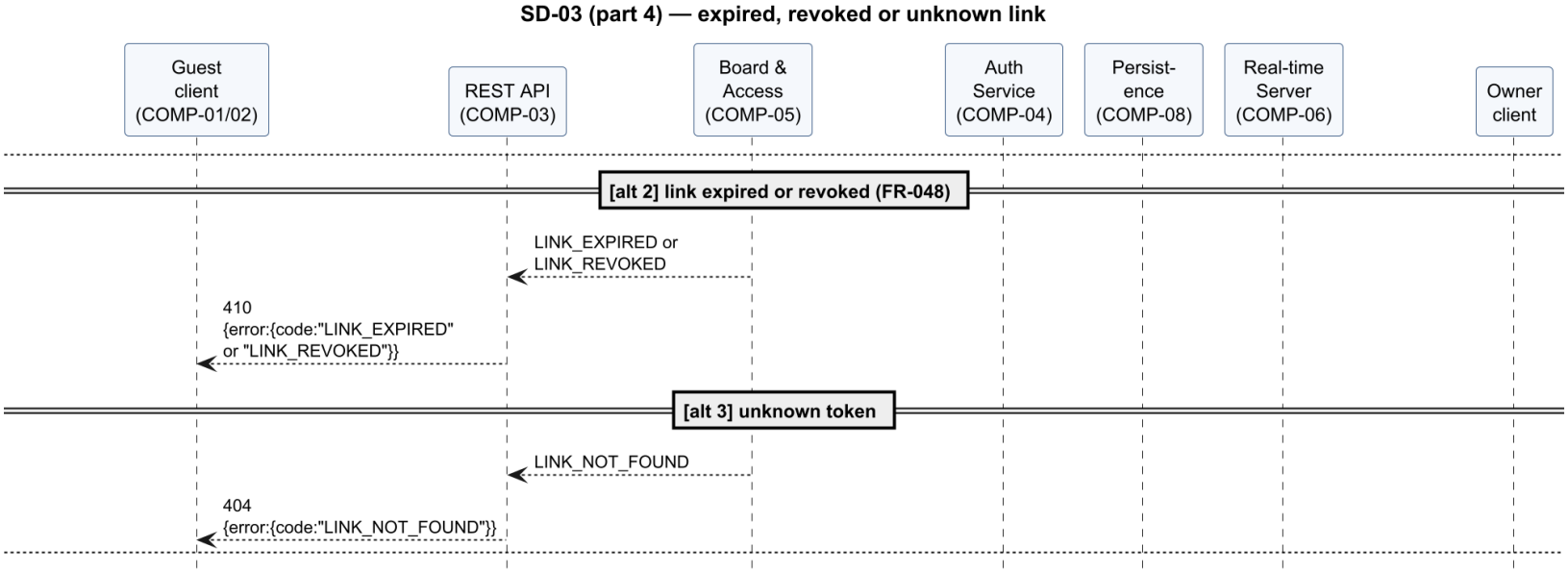


Figure 21 — SD-03 (part 4 of 5): expired, revoked or unknown link

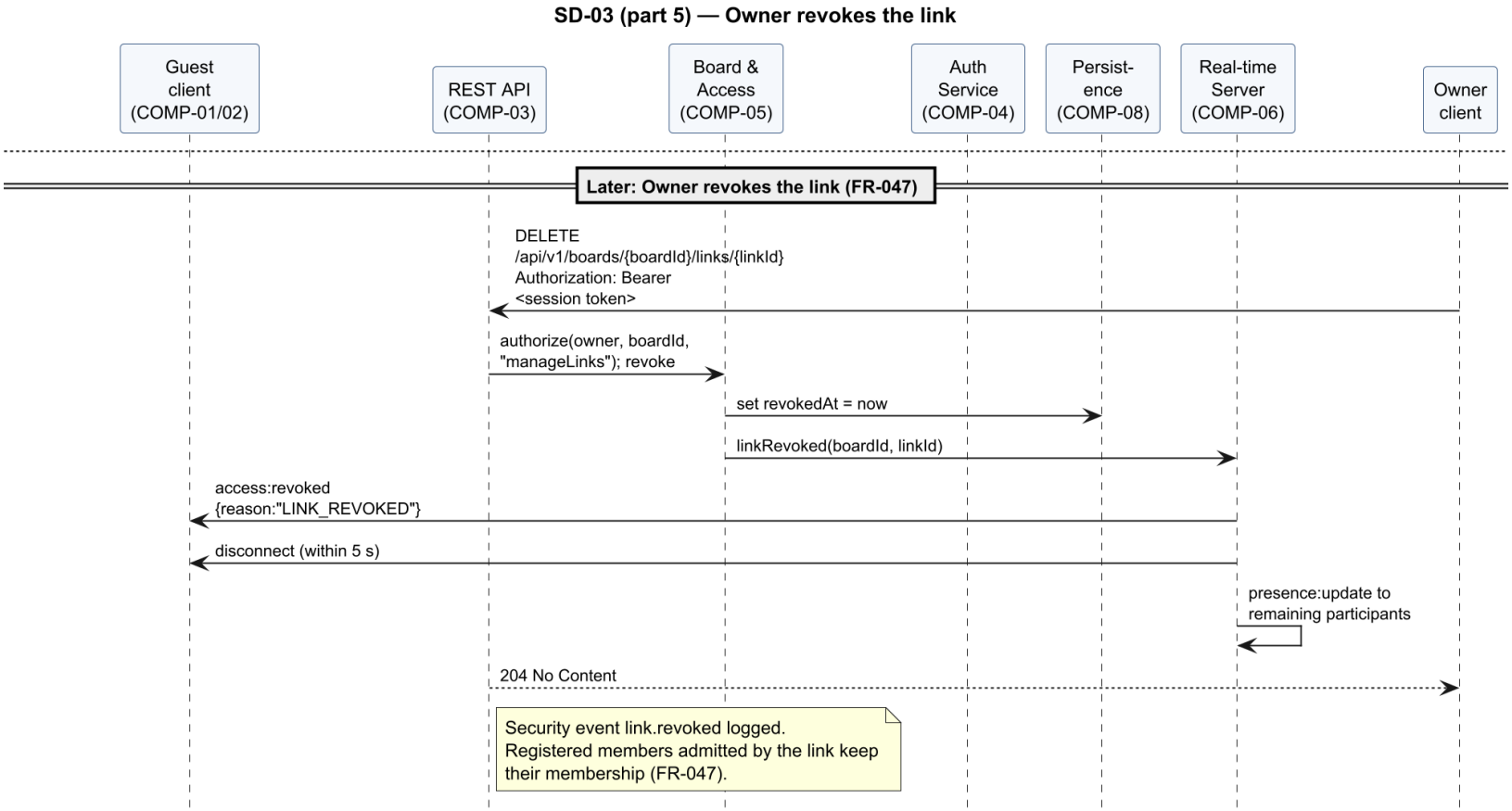


Figure 22 — SD-03 (part 5 of 5): Owner revokes the link

SD-04 — Transfer board ownership

Realises FR-050, FR-051, SEC-REQ-003, SEC-REQ-008. The Owner transfers ownership to an Editor or Viewer member through API-02 (`POST /api/v1/boards/{boardId}/transfer-ownership`). The Board & Access Service verifies that the requester is the Owner and that the target is eligible, updates the ownership and memberships, and the Real-time Collaboration Server pushes the new roles to both users. Requests without `confirm: true`, from non-Owners or guests, for a board the requester has no role on, for an ineligible target, or that lose a race with a concurrent transfer are rejected and change no role.

Figures 23–28 show the flow in 6 parts.

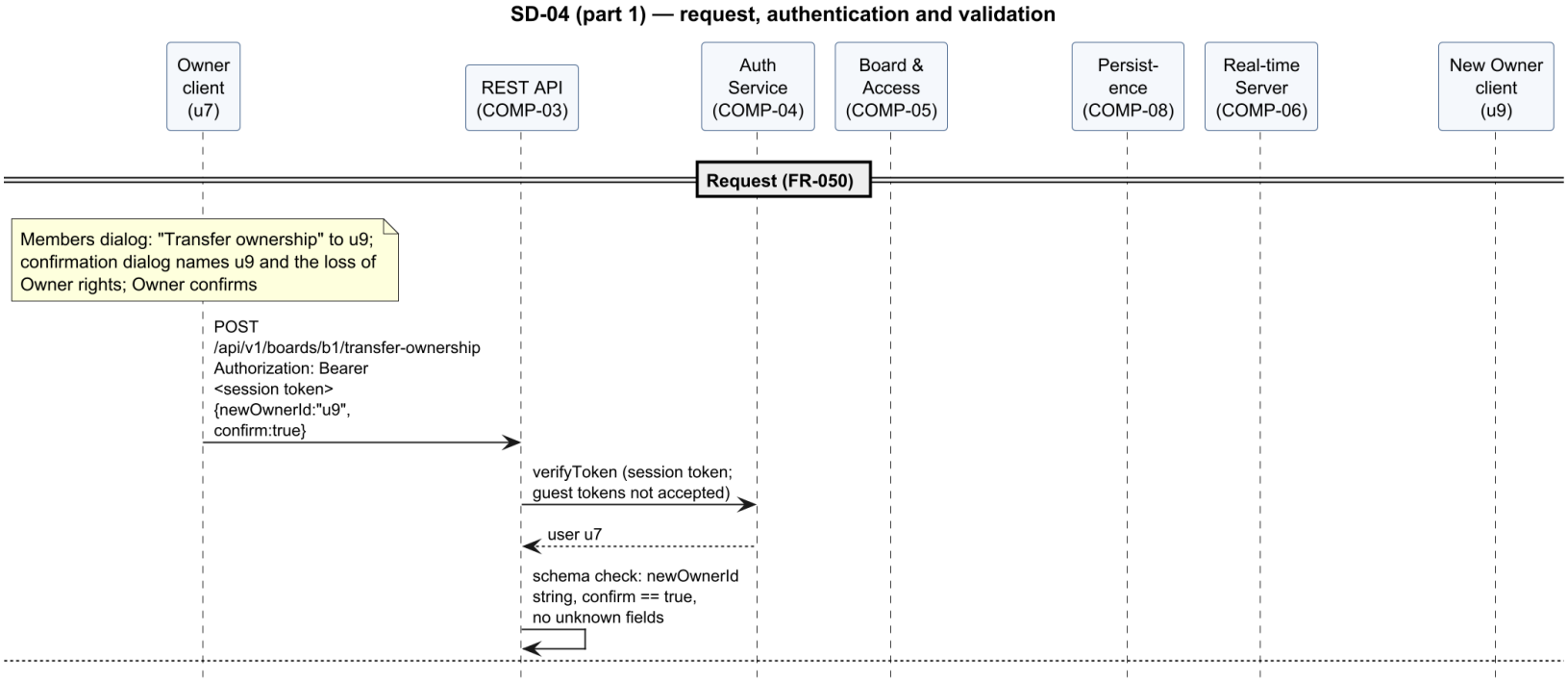


Figure 23 — SD-04 (part 1 of 6): request, authentication and validation

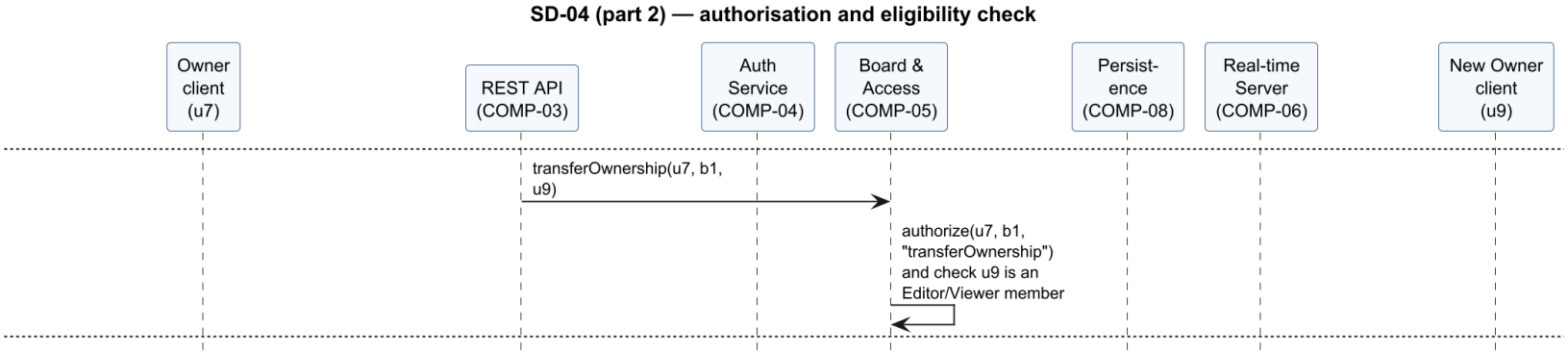


Figure 24 — SD-04 (part 2 of 6): authorisation and eligibility check

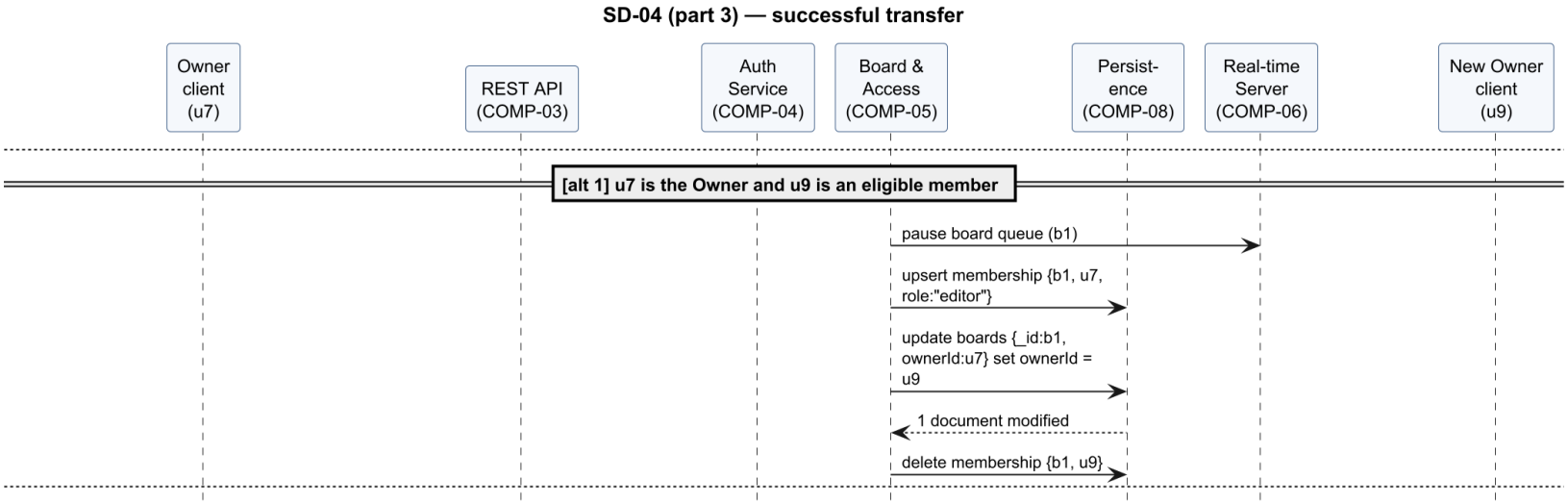


Figure 25 — SD-04 (part 3 of 6): successful transfer

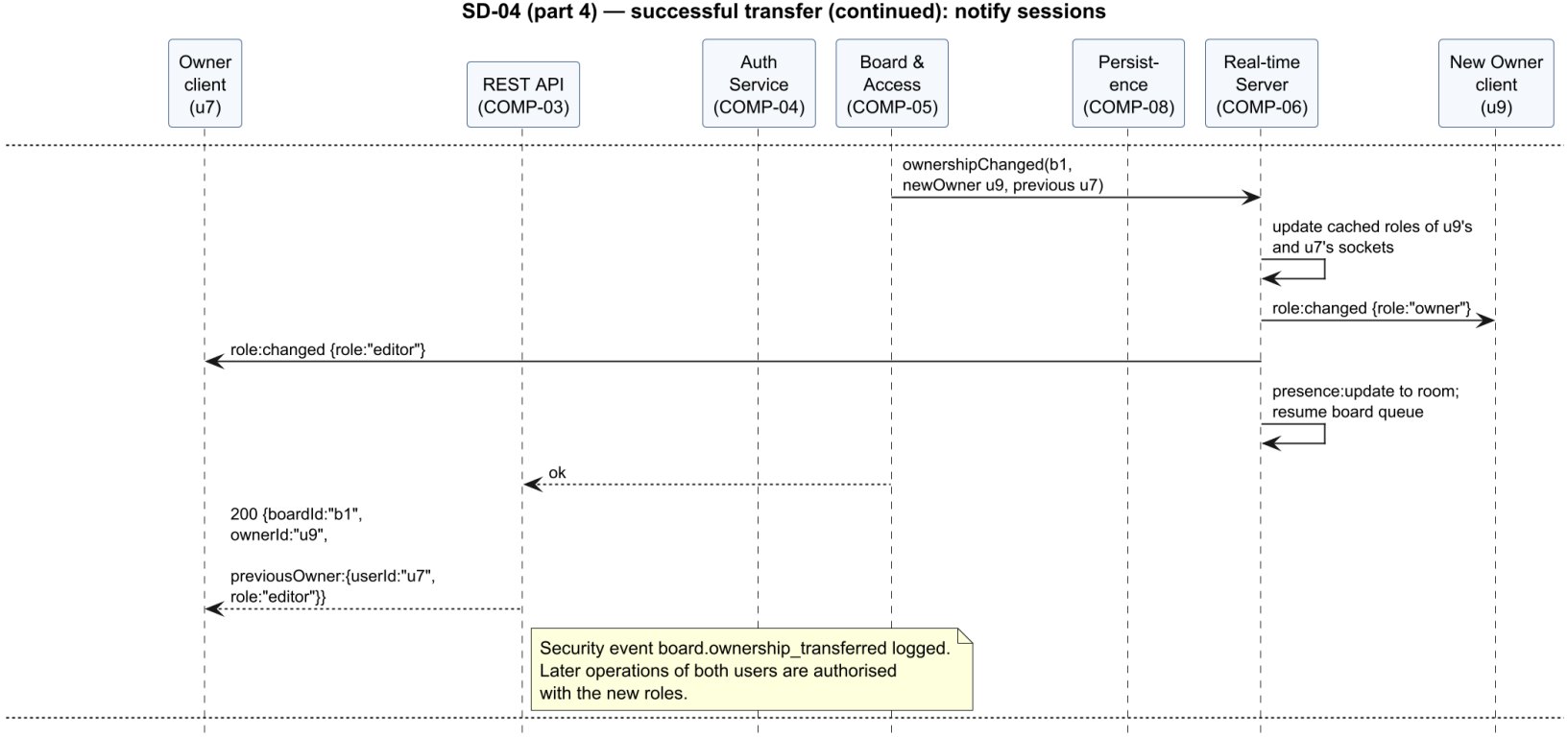


Figure 26 — SD-04 (part 4 of 6): successful transfer (continued): notify sessions

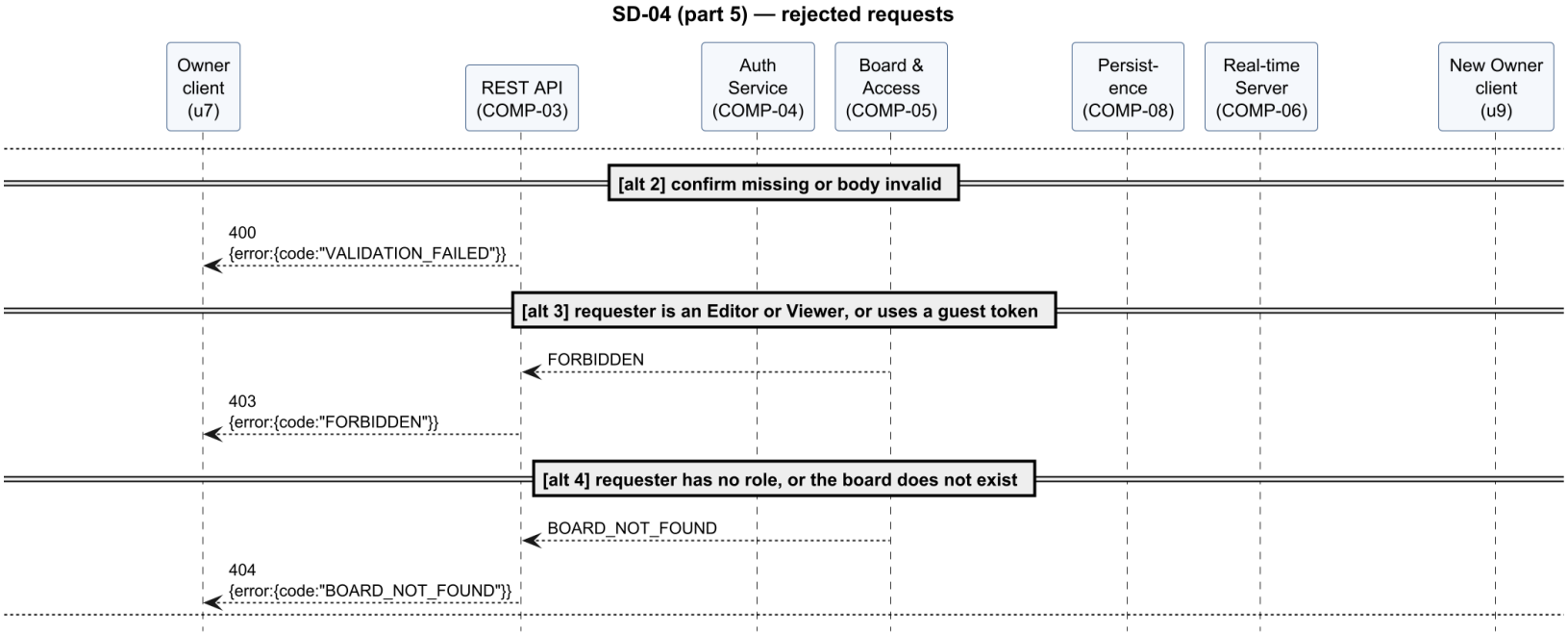


Figure 27 — SD-04 (part 5 of 6): rejected requests

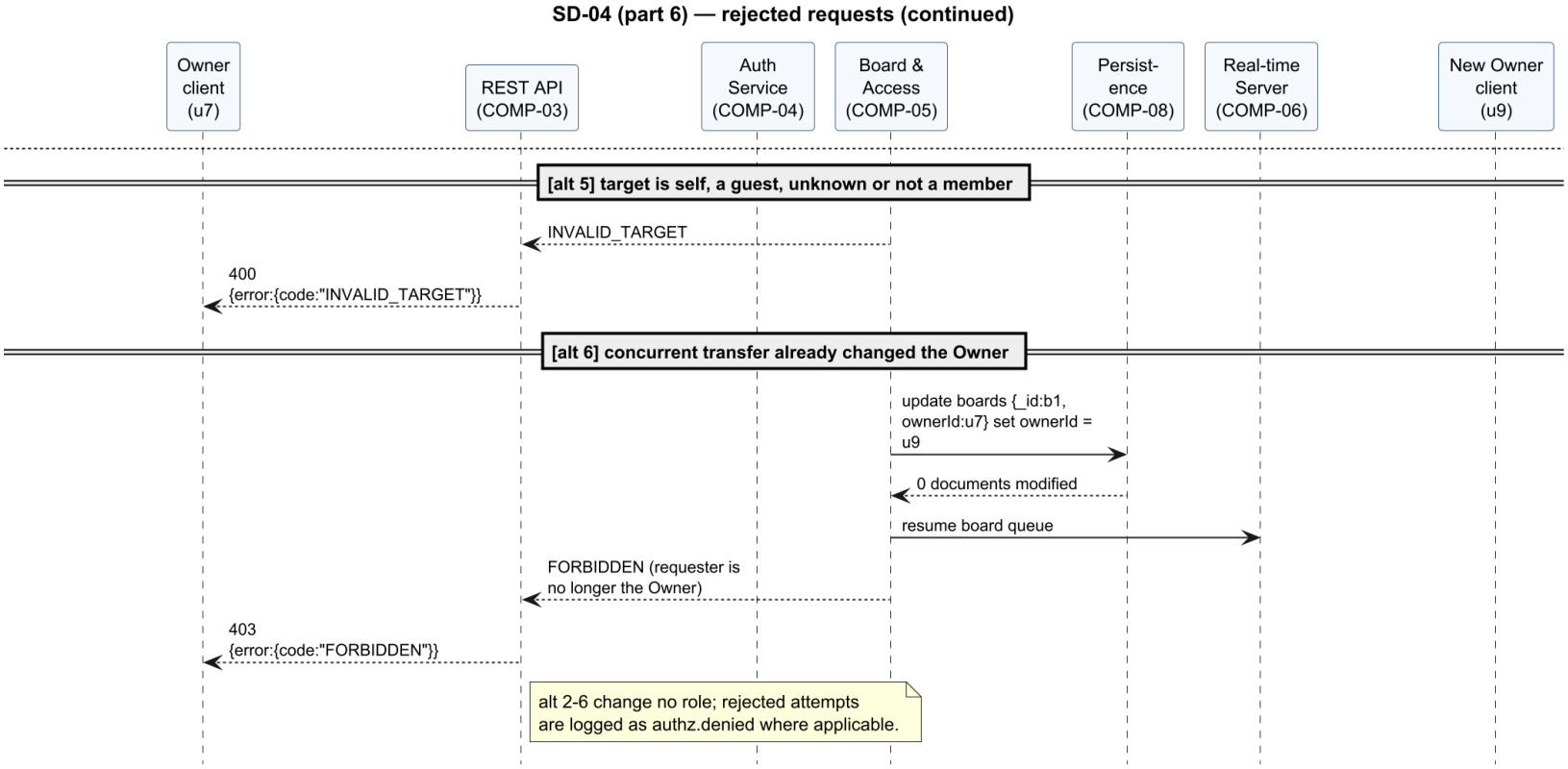


Figure 28 — SD-04 (part 6 of 6): rejected requests (continued)

SD-05 — Clear the board with interrupted-clear recovery

Realises FR-007, FR-030, FR-051. The Owner's `board:clear` runs as a task in the board queue: the server applies the "before clear" version trigger, commits the clear in the durable journal (`boards.pendingOp`), deletes the board objects and completes the journal entry with the new sequence number in one atomic update; only then is the clear acknowledged and broadcast. If the server stops after the commit, start-up recovery completes the clear before any client can reconnect (§3.6).

Figures 29–31 show the flow in 3 parts.

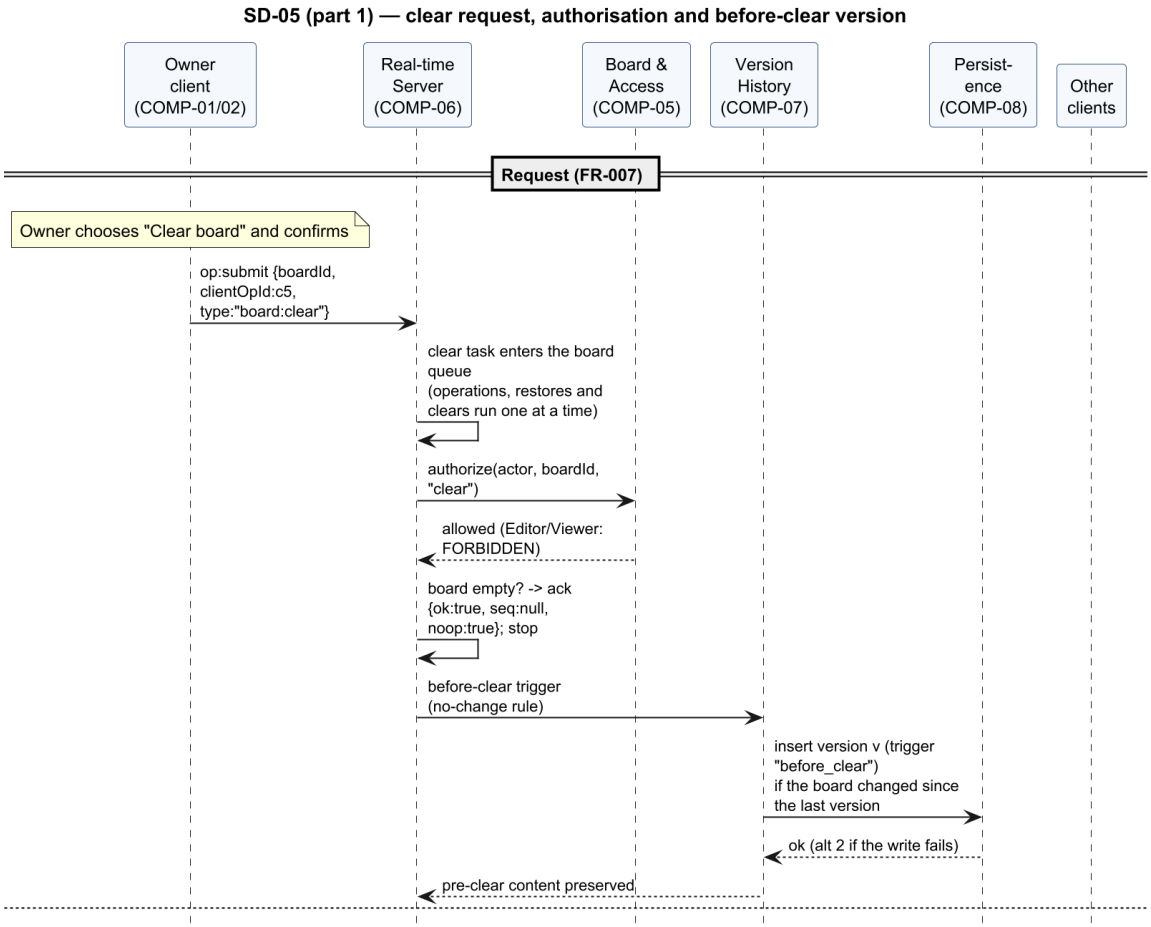


Figure 29 — SD-05 (part 1 of 3): clear request, authorisation and before-clear version

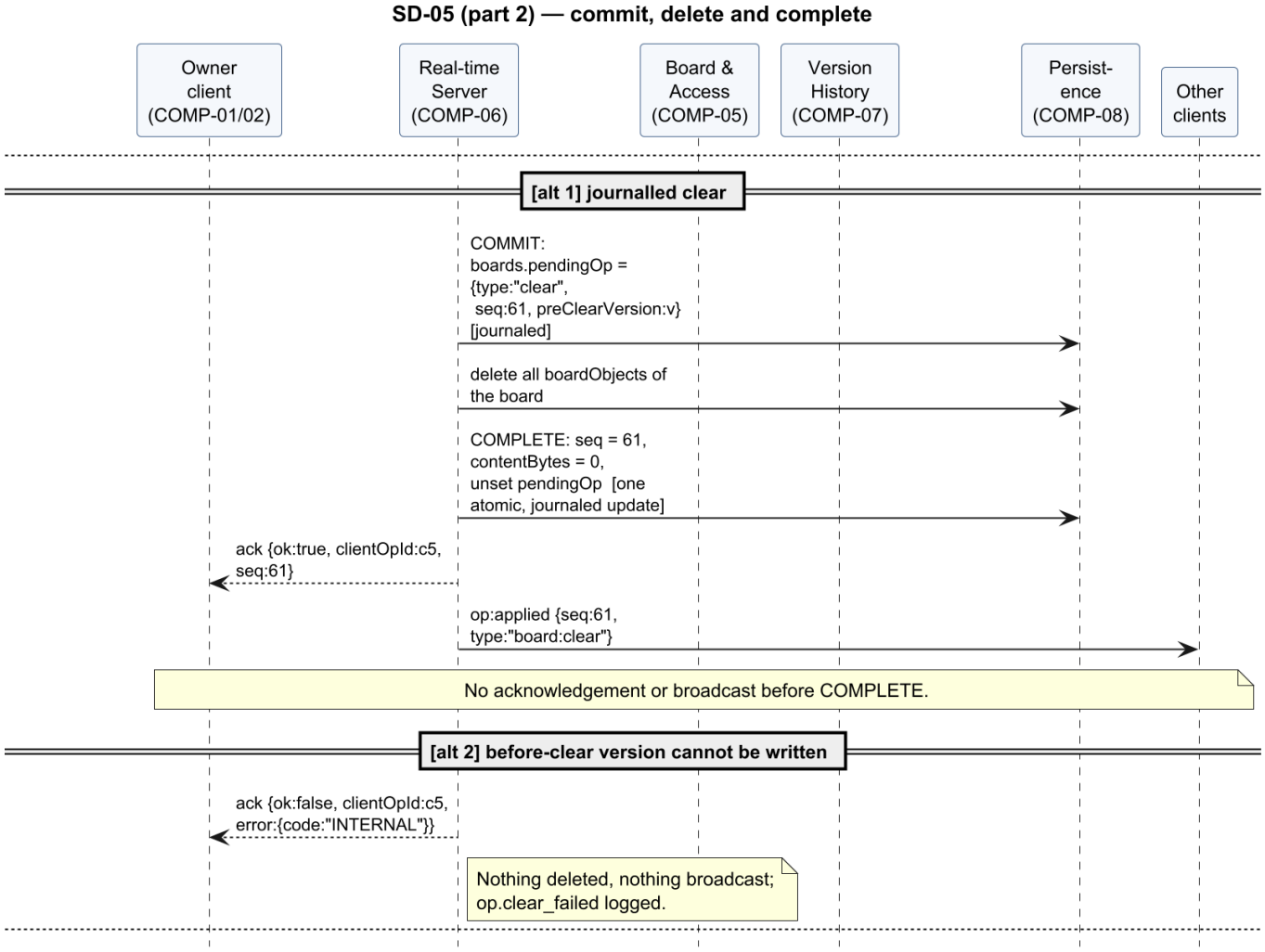


Figure 30 — SD-05 (part 2 of 3): commit, delete and complete

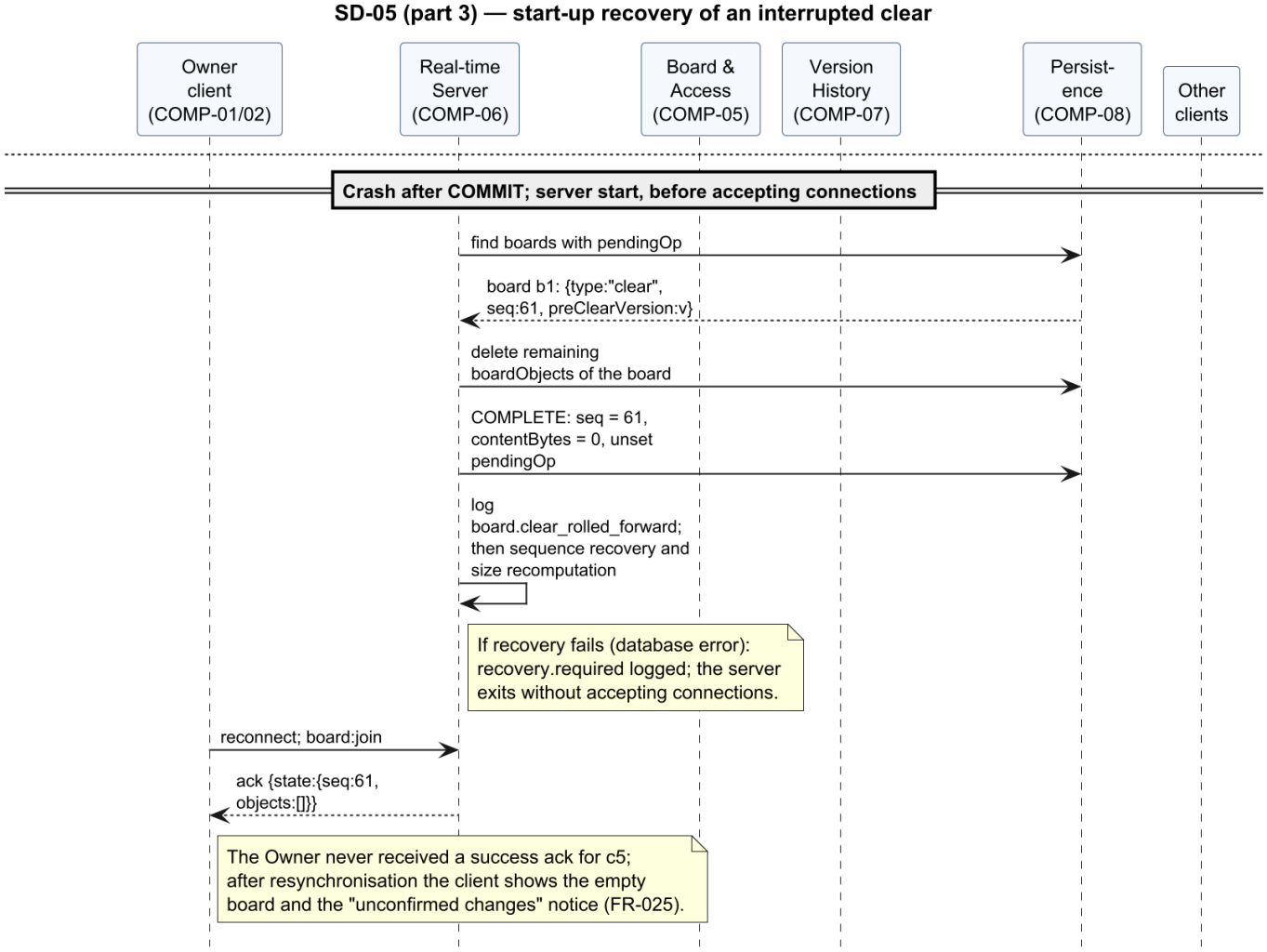


Figure 31 — SD-05 (part 3 of 3): start-up recovery of an interrupted clear

4.3 API Design

Three interfaces are specified (the template requires at least two components): API-01 for the Real-time Collaboration Server (COMP-06), API-02 for accounts, boards, links, members and ownership transfer (COMP-03/04/05) and API-03 for version history (COMP-03/07). REST and Socket.IO use the same tokens (SAD §3.9.2) and the same `authorize` policy (§3.9.3). All payloads are JSON. Placeholders such as `<token>` stand for real values and are never real secrets.

Common error object (REST bodies and Socket.IO acknowledgements): `{"error": {"code": "<CODE>", "message": "<human-readable text>", "requestId": "<id>"}}`

Code	HTTP status (REST)	Meaning
`VALIDATION_FAILED`	400	Payload does not match the schema or exceeds limits, or a required confirmation is missing (SEC-REQ-005)
`INVALID_TARGET`	400	Ownership-transfer target is not an eligible registered member (FR-050)
`UNAUTHENTICATED`	401	Missing, invalid, expired or revoked token (SEC-REQ-002)
`FORBIDDEN`	403	Role does not permit the action (FR-051, SEC-REQ-003)
`BOARD_NOT_FOUND`, `VERSION_NOT_FOUND`, `LINK_NOT_FOUND`, `OBJECT_NOT_FOUND`	404	Target does not exist (also used when the requester may not know it exists)
`EMAIL_TAKEN`	409	Registration with an existing e-mail (FR-040)
`BOARD_FULL`	— (Socket.IO only)	Operation would exceed the 15 MiB BSON board content limit (FR-008); rejected before anything is stored
`IDEMPOTENCY_KEY_REUSED`	409	A restore `Idempotency-Key` was reused for a different version (API-03)
`LINK_EXPIRED`, `LINK_REVOKED`	410	Share link no longer valid (FR-047, FR-048)
`RATE_LIMITED`	429	Rate limit exceeded (SEC-REQ-006)
`INTERNAL`	500	Unexpected server error; details only in server log

API-01 — Real-time board events (Socket.IO, COMP-06)

Realises FR-001–FR-008 (operations), FR-020–FR-026, FR-034, FR-047–FR-050, FR-052, SEC-REQ-002, SEC-REQ-003, SEC-REQ-005, SEC-REQ-006.

Connection. Namespace `/board`. Handshake: `io("/board", { auth: { token: "<session or guest token>" } })`. Invalid token → `connect_error` with `data.code = "UNAUTHENTICATED"`. Maximum message size 65,536 bytes per packet (measure M1, SAD §3.6).

Client → server events (every event uses a Socket.IO acknowledgement callback):

Event	Payload	Acknowledgement (success)	Errors
`board:join`	`{boardId}`	`{ok: true, role, state: {seq, objects: [...]}, participants: [{id, displayName, role, guest}]}`	`FORBIDDEN`, `BOARD_NOT_FOUND`, `LINK_EXPIRED`, `LINK_REVOKED`
`board:leave`	`{boardId}`	`{ok: true}`	—
`op:submit`	`{boardId, clientOpId, type, payload}` with `type` ∈ `object:add` (payload = full object), `object:update` (`{id, ...changedProperties}` with complete new values), `object:delete` (`{id}`), `board:clear` (`{}`, Owner only)	`{ok: true, clientOpId, seq}`; for `board:clear` on an empty board `{ok: true, clientOpId, seq: null, noop: true}`	`{ok: false, clientOpId, error}` with `VALIDATION_FAILED`, `FORBIDDEN`, `OBJECT_NOT_FOUND`, `BOARD_FULL`, `RATE_LIMITED`, `INTERNAL`

Server → client events:

Event	Payload	When
`op:applied`	`{boardId, seq, clientOpId, actor: {id, displayName}, type, payload}`	After every accepted operation, to all sockets in the room except the submitter (who receives the acknowledgement)
`board:state`	`{boardId, seq, objects: [...]}`	After a restore, to every socket in the room (FR-034); the client replaces `confirmed` and `lastSeq` and keeps its pending operations. Gap repair uses the `board:join` acknowledgement instead (§4.1.3)
`version:restored`	`{boardId, restoredFrom, newVersion, actor}`	After a restore, for the version panel and notification
`version:created`	`{boardId, number, trigger, createdAt}`	After any new version (version panel refresh)
`presence:update`	`{boardId, participants: [{id, displayName, role, guest}]}`	On join, leave, disconnect, role change
`role:changed`	`{boardId, role}`	When the Owner changes this member's role (FR-049), and to both users after an ownership transfer (FR-050); the client updates its enabled tools immediately
`access:revoked`	`{boardId, reason}` with `reason` ∈ `LINK_REVOKED`, `LINK_EXPIRED`, `MEMBER_REMOVED`, `BOARD_DELETED`	Immediately before the server disconnects the socket (FR-044, FR-047–FR-049)
`board:renamed`	`{boardId, title}`	After the Owner renames the board through `PATCH /boards/{boardId}` (API-02; added 02-11-2026, issue #13); the board page updates its title without a reload
`notice`	`{boardId, code: "VERSION_NOT_SAVED", message}`	To the Owner's sockets when an automatic version could not be stored because its snapshot is too large (defensive check, §3.6). **Known gap (02-11-2026):** the client sync module does not subscribe to this event, so the notice is currently not shown

Example. Each line shows the direction and event name, followed by the complete JSON payload of that message.

```
client → op:submit {"boardId":"b1","clientOpId":"c1","type":"object:add",
                    "payload":{"id":"o1","kind":"rect","x":10,"y":20,"w":120,"h":60,
                               "color":"#1E88E5","strokeWidth":3}}
server → ack       {"ok":true,"clientOpId":"c1","seq":41}
server → room      op:applied {"boardId":"b1","seq":41,"clientOpId":"c1",
                    "actor":{"id":"u7","displayName":"Rachit"},
                    "type":"object:add",
                    "payload":{"id":"o1","kind":"rect","x":10,"y":20,"w":120,"h":60,
                               "color":"#1E88E5","strokeWidth":3}}
```

Ordering and delivery guarantees (details in §4.1.2–§4.1.4): the server processes operations of one board strictly in queue order, persists each before acknowledging or broadcasting it, and emits acknowledgements and `op:applied` events in sequence order. Clients apply events strictly by `seq`, ignore duplicates, buffer out-of-order events and resynchronise with `board:join` when a gap is not closed within 1 s or after reconnection.

API-02 — Accounts, boards, share links, members and ownership (REST, COMP-03/04/05)

Realises FR-040–FR-051, SEC-REQ-001, SEC-REQ-002, SEC-REQ-003, SEC-REQ-004, SEC-REQ-005, SEC-REQ-006, SEC-REQ-008. Base path `/api/v1`. Authenticated endpoints require `Authorization: Bearer <token>`. "Role" is the minimum board role required (checked by COMP-05). **403 vs 404 rule:** a requester with no role on the board receives `404 BOARD_NOT_FOUND` (the board's existence is not revealed); a requester with a role that is too low receives `403 FORBIDDEN`.

Method and path	Auth / role	Request body	Success response	Errors
`POST /auth/register`	none	`{email, password, displayName}`	`201 {userId}`	400 `VALIDATION_FAILED`, 409 `EMAIL_TAKEN`
`POST /auth/login`	none	`{email, password}`	`200 {token, expiresAt, user: {id, displayName}}`	401 `UNAUTHENTICATED`, 429 `RATE_LIMITED`
`POST /auth/logout`	registered user	—	`204`	401
`GET /boards`	registered user	—	`200 [{id, title, role, updatedAt}]`	401
`POST /boards`	registered user	`{title}`	`201 {id, title, role: "owner"}`	400, 401
`GET /boards/{boardId}`	Viewer	—	`200 {id, title, ownerId, role, updatedAt}`	401, 403, 404

Method and path	Auth / role	Request body	Success response	Errors
PATCH /boards/{boardId} (rename)	Owner	{title} — 1–100 characters, not blank, no other fields (same schema as POST /boards); stored trimmed	200 {id, title, role, updatedAt}; COMP-06 sends board:renamed {boardId, title} to the board's room	400 VALIDATION_FAILED, 401, 403, 404
DELETE /boards/{boardId}	Owner	—	204	401, 403, 404
POST /boards/{boardId}/links	Owner	{role: "viewer" | "editor", expiresIn: "1h" | "24h" | "7d"}	201 {linkId, url, role, expiresAt} (url contains the token; shown once)	400, 401, 403, 404
GET /boards/{boardId}/links	Owner	—	200 [{linkId, role, expiresAt, createdAt}] (no tokens)	401, 403, 404
DELETE /boards/{boardId}/links/{linkId}	Owner	—	204 (revokes; COMP-06 disconnects affected guests)	401, 403, 404
POST /links/resolve	link token in body; bearer session token optional	{token, displayName?} (displayName required when no session token is sent)	With a valid session token: 200 {boardId, role} and membership created or kept (FR-046); without: 200 {boardId, role, guestToken, expiresAt}	400, 404 LINK_NOT_FOUND, 410 LINK_EXPIRED/LINK_REVOKED, 429
GET /boards/{boardId}/members	Viewer	—	200 [{userId, displayName, role}]	401, 403, 404
PATCH /boards/{boardId}/members/{userId}	Owner	{role: "viewer" | "editor"}	200 {userId, role}	400, 401, 403, 404
DELETE /boards/{boardId}/members/{userId}	Owner	—	204	401, 403, 404
POST /boards/{boardId}/transfer-ownership	Owner (session token)	{newOwnerId, confirm: true}	200 {boardId, ownerId, previousOwner: {userId, role: "editor"}}	400 VALIDATION_FAILED/INVALID_TARGET, 401, 403, 404 (details below)
GET /health	none	—	200 {status: "ok", db: "up"}	503 if the database is down

Board rename (added 02-11-2026, implemented in issue #13 / PR #28). The endpoint runs in the board's queue and re-checks the requester's role there, so a rename cannot race with a deletion or an ownership transfer. The policy action `rename` is Owner-only (`server/src/access/policy.js`), like board deletion. Rename is not covered by an SRS requirement or by the FR-051 matrix (SRS §2.7, scope decision pending); it is logged as the operational event `board.renamed`, not as a security event. Tests: `server/tests/integration/boards.test.js`, `e2e/boards.spec.cjs`.

Example.

```
POST /api/v1/boards/b1/links
Authorization: Bearer <session token>
{"role":"editor","expiresIn":"24h"}

201 Created
{"linkId":"l3","url":"https://<host>/join#<token>","role":"editor",
 "expiresAt":"2026-10-11T18:30:00Z"}
```

Transfer ownership — POST /api/v1/boards/{boardId}/transfer-ownership

Realises FR-050 (with FR-051, SEC-REQ-003, SEC-REQ-008). Flow: SD-04. Tests: TC-018, TC-102.

Aspect	Specification
Authentication	Bearer **session** token of a registered user. A guest token is rejected with `403 FORBIDDEN` (guests can never hold the Owner role). Missing, invalid or expired token → `401 UNAUTHENTICATED`.
Authorisation	The requester must be the board's current Owner (`boards.ownerId`). Editor or Viewer → `403 FORBIDDEN`; no role on the board → `404 BOARD_NOT_FOUND`.
Request body	`{"newOwnerId": "<userId>", "confirm": true}`. Unknown fields are rejected.
Safeguards	The client shows a confirmation dialog naming the new Owner and the loss of Owner rights; the API additionally requires `confirm: true` (missing or not `true` → `400 VALIDATION_FAILED`). The update is conditional on the requester still being the Owner, so two concurrent transfers cannot both succeed (the second receives `403`).
Target validation	`newOwnerId` must identify a registered user who currently has an Editor or Viewer membership on this board. Self, unknown user IDs, non-members and guest IDs → `400 INVALID_TARGET`.
Effect (in this order)	1. Board queue paused. 2. Upsert membership of the requester with role `editor`. 3. Conditional update `boards.ownerId: requester → newOwnerId`. 4. Delete the new Owner's membership row (the Owner is stored on the board only). 5. Queue resumed. A crash after step 2 leaves the requester as Owner (the Owner check precedes membership lookup); a crash after step 3 leaves a redundant membership row for the new Owner that is ignored and removed on the next membership change.
Success response	`200 {"boardId": "b1", "ownerId": "<newOwnerId>", "previousOwner": {"userId": "<requesterId>", "role": "editor"}}`
Notifications	COMP-06 sends `role:changed {role: "owner"}` to the new Owner's sockets and `role:changed {role: "editor"}` to the former Owner's sockets, then `presence:update` to the room. Both dashboards show the new roles on next load.
Logging	Security event `board.ownership_transferred` (§4.4): `actorId` = former Owner, `subjectId` = new Owner, `boardId`, time, request ID, `outcome` (SEC-REQ-008).
Errors	`400 VALIDATION_FAILED` (body invalid or `confirm` missing), `400 INVALID_TARGET`, `401 UNAUTHENTICATED`, `403 FORBIDDEN` (not Owner, or guest token), `404 BOARD_NOT_FOUND` (board missing or requester has no role), `500 INTERNAL`.

```
POST /api/v1/boards/b1/transfer-ownership
Authorization: Bearer <session token>
{"newOwnerId":"u9","confirm":true}

200 OK
{"boardId":"b1","ownerId":"u9","previousOwner":{"userId":"u7","role":"editor"}}
```

API-03 — Version history (REST, COMP-03/07)

Realises FR-030–FR-035, FR-051, NFR-005. Base path `/api/v1`; bearer token required (session or guest token for the same board).

Method and path	Role	Request	Success response	Errors
`GET /boards/{boardId}/versions`	Viewer	query `limit` (1–50, default 20), `before` (version number, for paging)	`200 [{number, trigger, restoredFrom, createdAt, createdBy}]`, newest first	401, 403, 404
`GET /boards/{boardId}/versions/{number}`	Viewer	—	`200 {number, trigger, createdAt, objects: [...]}` (read-only preview, FR-032)	401, 403, 404 `VERSION_NOT_FOUND` (never existed or removed by retention, FR-035)
`POST /boards/{boardId}/versions/{number}/restore`	Owner	header `Idempotency-Key: <UUID>` (required, one per user action)	`201 {newVersion, restoredFrom, seq}`; creates a new version, even if the content equals the current board; COMP-06 then emits `board:state` and `version:restored` to the room (SD-02). A retry with the same key and version returns the recorded `201` body (same `newVersion`, `restoredFrom`, `seq`) without restoring again or creating another version; a new key is a new restore	400 (missing or malformed key), 401, 403, 404 `BOARD_NOT_FOUND`/`VERSION_NOT_FOUND`, 409 `IDEMPOTENCY_KEY_REUSED` (key already used for a different version), 500 `INTERNAL` (no success: before the commit point nothing changed; after it the restore is completed by recovery, and a retry with the same key reports the outcome)

`trigger` ∈ `periodic`, `before_clear`, `last_left`, `restore`. Automatic versions follow the no-change rule of SRS §4.3 (D5b). Versions are created only by the server (FR-030); there is no endpoint to create, edit or delete a version directly, which protects history integrity (SEC-OBJ-002).

Restore semantics (journal, commit point, recovery, serialisation and the key-retention limit) are specified in SAD §3.6.

Example.

```
POST /api/v1/boards/b1/versions/12/restore
Authorization: Bearer <session token>
Idempotency-Key: 6f1c2a9e-3b7d-4e21-9a55-0c8d4f2b7e10

201 Created
{"newVersion":19,"restoredFrom":12,"seq":88}

Retry of the same request with the same key (recorded result; nothing restored again):
201 Created
{"newVersion":19,"restoredFrom":12,"seq":88}

Same key reused for a different version:
POST /api/v1/boards/b1/versions/15/restore
Authorization: Bearer <session token>
Idempotency-Key: 6f1c2a9e-3b7d-4e21-9a55-0c8d4f2b7e10

409 Conflict
{"error":{"code":"IDEMPOTENCY_KEY_REUSED",
          "message":"This Idempotency-Key was already used to restore version 12",
          "requestId":"<id>"}}
```

4.4 Error Handling, Logging & Monitoring

Error handling principles

- Every error returned to a client uses the common error object (§4.3) with a stable `code`; messages never contain stack traces, database details, tokens or other users' data.
- REST: a central Express error middleware maps errors to the HTTP statuses in §4.3; unknown errors become `500 INTERNAL` and are logged with the request ID.
- Socket.IO: handlers never throw to the client; they acknowledge `{ok: false, clientOpId, error}`. The connection stays open after operation-level errors; it is closed only for authentication failure, revocation, removal, board deletion (each preceded by `access:revoked`) or by the client after an acknowledgement timeout (§4.1.3).
- Persistence first: a database error while applying an operation causes rejection with `INTERNAL`, no acknowledgement of success, no broadcast and no change of `seq`, so clients never see an operation that was not stored (FR-027).
- Restore, clear and ownership transfer pause the board queue while they change state, so no operation is applied to a half-changed board (SD-02, SD-04).
- Snapshot too large (defensive, should not occur because of FR-008): the version is not written and `version.snapshot_too_large` is logged. Periodic or last-participant trigger: the Owner's client shows "This version could not be saved" and the live board is unaffected. Before-clear trigger: the clear is rejected (`INTERNAL`). Restore: the restore is rejected (`INTERNAL`).
- Version write failure (database error) follows the same rules: a clear or restore is never completed without its version (FR-007, FR-033). After its commit point a clear or

restore is completed by the running task or by start-up recovery and is never reported as successful before completion (§3.6).

Client reactions (COMP-01/02)

Situation	Client behaviour
`VALIDATION_FAILED`, `FORBIDDEN`, `OBJECT_NOT_FOUND`, `BOARD_FULL` on an operation	Remove the operation from `pending`, recompute the view, show a short message naming the reason (FR-026)
`RATE_LIMITED`	As above, with "You are sending changes too quickly"
`UNAUTHENTICATED`	Go to the login page (registered user) or the guest entry page (guest)
`access:revoked`	Show the reason (link revoked or expired, removed, board deleted) and leave the board
`role:changed`	Enable or disable tools for the new role without reload (FR-049, FR-050)
Connection lost	"Disconnected — reconnecting…" banner, tools disabled; after resynchronisation, notice if unconfirmed changes were discarded (FR-025)
REST `INVALID_TARGET` (transfer)	Keep the Members dialog open and explain that only Editor or Viewer members can become Owner

Logging (COMP-09)

- Format: one JSON object per line, written by COMP-09 to the server's standard output, which the test environment and a deployment redirect to an append-only log file (`logs/server.log`). Common fields: `timestamp` (ISO 8601, UTC), `level` (`error`, `warn`, `info`, `debug`), `event`, `requestId`, `boardId`, `actorId`, `outcome`, `errorCode` and `durationMs` where relevant.
- `requestId` is the correlation ID: a UUID generated by COMP-03 for every HTTP request (and returned in the error object, §4.3) and by COMP-06 for every Socket.IO event it handles; Socket.IO entries also carry `socketId`. Entries written by start-up recovery have `requestId: null`.
- Security events (SEC-REQ-008): `auth.login_failed`, `auth.rate_limited`, `authz.denied`, `link.created`, `link.revoked`, `link.expired_disconnect`, `member.role_changed`, `member.removed`, `board.ownership_transferred`, `version.restored`, `board.deleted`.
- Operational events: `op.rejected` (with `errorCode`), `board.resync_requested`, `version.created`, `version.snapshot_too_large`, `version.restore_failed`, `version.restore_rolled_forward`, `op.clear_failed`, `board.clear_rolled_forward`, `recovery.required`, `db.write_failed`.
- Never logged: passwords, session or guest tokens, share-link tokens, `Authorization` headers, the e-mail address entered at login, board object content or text. The allow-list above enforces this for security events, and a redaction function removes these fields from all other entries before writing; TC-104 searches the log for them (Pass on 12-10-2026; historical v1.0 note: planned when this was written).
- Default level `info` in test and deployment; `debug` only on developer machines.

Security event record (SEC-REQ-008, D13b). The component that detects the event creates the record and passes it to COMP-09 (`log.security`): COMP-04 for `auth.login_failed`; COMP-03 for `auth.rate_limited` and REST `authz.denied`; COMP-06 for Socket.IO `authz.denied` and `link.expired_disconnect`; COMP-05 for link, member, ownership and board-deletion events; COMP-07 for `version.restored`. COMP-09 writes only the fields below (an allow-list; any other field is dropped) and adds `timestamp`.

Field	Content	When not applicable
`event`	One of the security event types above	Always present
`timestamp`	ISO 8601, UTC, millisecond precision	Always present
`requestId`	Correlation ID of the HTTP request or Socket.IO event that caused the entry, plus `socketId` for Socket.IO; for an event raised by a server timer (link expiry), a newly generated UUID	Always present for security events
`actorType`, `actorId`	`user` with the account ID, or `guest` with the guest session ID, once a valid token or login has identified the requester	`actorType: "anonymous"`, `actorId: null` (failed login, missing or invalid token)
`boardId`	The board the event concerns	`null` (login events, rate limits not tied to a board)
`subjectId`	Internal account ID (`users._id`, never an e-mail address) of the account affected by the action: member whose role changed or who was removed, new Owner, or the registered account whose e-mail was entered at a failed login (COMP-04 looks the address up and writes only the ID)	`null` (no such account, or not relevant)
`outcome`, `reason`	`success` or `rejected`; for a rejection the error code of §4.3 (e.g. `UNAUTHENTICATED`, `FORBIDDEN`, `RATE_LIMITED`, `LINK_REVOKED`); a failed login always uses `UNAUTHENTICATED`, so the log does not say whether the e-mail exists	`reason: null` for successful events
`details`	Non-sensitive values only: `role`, `fromRole`, `toRole`, `linkId`, `restoredFrom`, `newVersion`	Omitted

Failed logins without account enumeration (SEC-REQ-006, SEC-REQ-008, FR-041). Every failed login returns the same response, `401 UNAUTHENTICATED` with one generic message, whether the e-mail is unknown or the password is wrong — the bodies differ only in the per-request `requestId` (§4.3); for an unknown e-mail COMP-04 still runs one bcrypt comparison against a fixed dummy hash, a design measure intended to keep the response times of both cases similar; it does not prove constant-time behaviour, and response timing has not been measured (it is not part of TC-108). The login limiter counts failures per entered e-mail address (lower-cased), registered or not, so the 6th failure returns `429 RATE_LIMITED` in both cases. The limiter keeps only an HMAC of the address (key from an environment variable) in memory for 15 minutes; the address itself is never stored or logged. Only the server log, which is readable by operators and not by clients, records `subjectId` for a registered account.

Monitoring metrics (written to the log every 10 s and exposed on a development-only metrics endpoint)

Metric	Purpose
`op_server_latency_ms` (receive → broadcast, per operation)	Server share of NFR-001 latency
`connected_sockets`, `participants_per_board`	Capacity (NFR-002)
`ops_rejected_total{code}`	Validation, authorisation, size and rate-limit rejections (SEC-REQ-003, -005, -006, FR-008)
`resyncs_total{reason}` (gap, reconnect), `resync_duration_ms`	Recovery behaviour (NFR-004)
`restore_duration_ms`	NFR-005
`snapshot_bytes`, `board_content_bytes`	Early warning for the 15 MiB board limit and 16 MiB document limit (§3.6)
`http_requests_total{status}`	Error rate of the REST API

`GET /api/v1/health` reports server and database status for test-environment checks (STP §6). Metrics support diagnosis only; NFR results are measured by the STP harness, not by these metrics.

4.5 UX Design

- **Layout:** top bar (board title, role badge, connection status, connected participants, Members button for every role; Share and Clear board for the Owner); left toolbar (select, pen, eraser, rectangle, ellipse, line, text, colour, stroke width); central canvas; right panel with tabs "Participants" and "History", which can be hidden from the top bar and overlays the canvas in windows 900 px wide or narrower (frontend redesign of 16-10-2026). Update 02-11-2026: the top bar also has an **Export PNG** button for every role (issue #16), and below 1024 px the labelled top-bar buttons become icon buttons with tooltips so that the title, role and status stay readable (issue #23, tested at 1280×720, 1366×768, 1920×1080 and the narrower breakpoints in `e2e/responsive.spec.cjs`). The dashboard offers **Rename** to the Owner (issue #13). Export and rename are outside the SRS version 1.0 baseline (SRS §2.7, scope decision pending).
- **Role awareness:** Viewers see no drawing tools and a "View only" badge; Editors do not see Clear, Share or Restore; Editors and Viewers can open Members as a read-only list, while role changes, removal and ownership transfer are shown only to the Owner (enforced by the server, SEC-REQ-003); disabled actions show a tooltip explaining why (NFR-007).
- **Feedback:** local strokes appear immediately (optimistic rendering); rejected changes disappear with a short message, including "Board is full" (FR-008, FR-026); a banner "Disconnected — reconnecting…" appears and tools are disabled while offline, followed by a notice if unconfirmed changes were discarded (FR-025); a notice is shown when the board is restored by the Owner (`version:restored`) or when access ends (`access:revoked`).
- **Destructive actions:** Clear board, Restore version, Delete board, Revoke link and Transfer ownership require a confirmation dialog naming the consequence (the transfer dialog names the new Owner and states that the current Owner becomes an Editor).

- **Version history panel:** list newest first with number, time and trigger (FR-031); selecting a version opens a read-only preview overlay with "Restore" (Owner only) and "Close" (FR-032).
- **Sharing dialog:** role selector (Viewer/Editor), expiry selector (1 h / 24 h / 7 days), "Create link", copy-to-clipboard; list of active links with revoke buttons (FR-045, FR-047).
- **Accessibility and viewport:** toolbar buttons have visible labels or tooltips and can be reached and activated with the keyboard (NFR-007); status is never shown by colour alone; the layout fits a 1280 px wide viewport without horizontally cut-off controls (NFR-006).

4.6 Open Issues & Next Steps

- Team sign-off of the v1.0 documents (DECISIONS.md E1).
- Done (12-10-2026): implementation of the system and of the test assets (generator, scripted clients, fault-injection hook), execution of the STP and measured verification of NFR-001 to NFR-005 (`docs/IMPLEMENTATION_STATUS.md`).
- Done (17-10-2026): TC-021 (NFR-006, NFR-007), Pass with limitations recorded in the STP TC-021 record.
- Open: the TC-009 offline check in the current stable Mozilla Firefox is not verified (the automated driver cannot simulate the outage there; TC-009 itself is Pass from Chromium, Chrome and Edge runs; not part of TC-021), and TC-109 on a deployed HTTPS instance (SEC-REQ-007, DECISIONS.md E3).
- Done (from 01-11-2026): the GitHub Actions workflows `app-ci` (lint, unit, integration, build, Chromium E2E) and `docs-check` (traceability) run on every Pull Request and push to `main`; both pass on `main` after PR #33 (DECISIONS.md E4).
- Done (02-11-2026): final validation of `main` (`docs/reports/RELEASE_READINESS_2026-11-02.md`): restore without `crypto.randomUUID()` (plain HTTP), forced-colours swatches, lost pointer capture and the top-bar layout were fixed; stable Mozilla Firefox 159 was exercised with Playwright's experimental BiDi driver (22 E2E tests passed; the TC-009 outage, double-click text editing and the PNG download remain manual checks there). Apple Safari has not been tested; WebKit results are engine evidence only.
- Open: the server's `notice` event (`VERSION_NOT_SAVED`, API-01) is not displayed by the client (no subscription in COMP-02).
- Future work (not in version 1.0): idempotent resubmission of unacknowledged real-time operations using `clientOpId`; replay of missed operations instead of full resynchronisation; several server instances with a Socket.IO adapter and a distributed lock for the board queue; chunked snapshot storage (e.g. GridFS) for boards above 15 MiB; CRDT-based merging for text; undo/redo across users; live cursors; export to PDF (PNG export exists, see §4.5).

5. Appendices

5.1 Glossary: see SRS §1.4 and SAD §1.4 (Board, Board object, Operation, Participant, Share link, Version, Sequence number, LWW, ADR, JWT, Room, STRIDE, SPA, ODM).

5.2 References: ISO/IEC/IEEE 42010 (architecture description); OWASP Top 10 and OWASP Cheat Sheet Series (authentication, session management, input validation); NIST SP 800-160 (systems security engineering); Socket.IO, Express.js, MongoDB and React official documentation.

5.3 Tools: PlantUML (all UML diagrams; sources in `docs/diagrams/src/`), Markdown working sources assembled into the supplied Word template by `tools/build_docs.py`, and `tools/check_trace.py` for ID and traceability checks, `tools/size_model.py` for the size calculation, and `tools/export_pdf.ps1` for the PDF export.

5.4 Architecture decision summary:

ADR	Decision	Status
ADR-1	Client–server, layered server, Socket.IO rooms, server-authoritative state (§3.5)	Selected (D10, D12)
ADR-2	MERN + Socket.IO technology stack (§3.6)	Selected (D10)
ADR-3	Per-property last-write-wins in server sequence order; persist before acknowledge/broadcast; client reconciliation and full-state resync (§4.1.2–§4.1.4)	Selected (D6, D12)
ADR-4	Full snapshots for versions with the no-change rule; restore always creates a new version; 15 MiB board content limit; durable per-board journal for clear and restore (§3.6)	Selected (D5, D5b, D11, D11c)
ADR-5	JWT sessions, bcrypt, hashed 256-bit share-link tokens in URL fragment (§3.9)	Selected (D13)